

Parallel program development

7

In the last four chapters, we haven't just learned about parallel APIs; we've also developed a number of small parallel programs, and each of these programs has involved the implementation of a parallel algorithm. In this chapter, we'll look at a couple of larger examples: solving n -body problems and implementing a sorting algorithm called sample sort. For each problem, we'll start by looking at a serial solution and examining modifications to the serial solution. As we apply Foster's methodology, we'll see that there are some striking similarities between developing shared-memory, distributed-memory, and CUDA programs. We'll also see that in parallel programming there are problems that we need to solve for which there is no serial analog. We'll see that there are instances in which, as parallel programmers, we'll have to start "from scratch."

7.1 Two n -body solvers

In an n -body problem, we need to find the positions and velocities of a collection of interacting particles over a period of time. For example, an astrophysicist might want to know the positions and velocities of a collection of stars, while a chemist might want to know the positions and velocities of a collection of molecules or atoms. An n -body solver is a program that finds the solution to an n -body problem by simulating the behavior of the particles. The input to the problem is the mass, position, and velocity of each particle at the start of the simulation, and the output is typically the position and velocity of each particle at a sequence of user-specified times, or simply the position and velocity of each particle at the end of a user-specified time period.

Let's first develop a serial n -body solver. Then we'll try to parallelize it for both shared-memory systems, distributed-memory systems, and GPUs.

7.1.1 The problem

For the sake of explicitness, let's write an n -body solver that simulates the motions of planets or stars. We'll use Newton's second law of motion and his law of universal gravitation to determine the positions and velocities. Thus, if particle q has position $\mathbf{s}_q(t)$ at time t , and particle k has position $\mathbf{s}_k(t)$, then the force on particle q exerted

by particle k is given by

$$\mathbf{f}_{qk}(t) = -\frac{Gm_qm_k}{|\mathbf{s}_q(t) - \mathbf{s}_k(t)|^3} [\mathbf{s}_q(t) - \mathbf{s}_k(t)]. \quad (7.1)$$

Here, G is the gravitational constant ($6.673 \times 10^{-11} \text{ m}^3/(\text{kg} \cdot \text{s}^2)$), and m_q and m_k are the masses of particles q and k , respectively. Also, the notation $|\mathbf{s}_q(t) - \mathbf{s}_k(t)|$ represents the distance from particle k to particle q . Note that in general the positions, the velocities, the accelerations, and the forces are vectors, so we're using boldface to represent these variables. We'll use an italic font to represent the other, scalar variables, such as the time t and the gravitational constant G .

We can use Formula (7.1) to find the total force on any particle by adding the forces due to all the particles. If our n particles are numbered $0, 1, 2, \dots, n-1$, then the total force on particle q is given by

$$\mathbf{F}_q(t) = \sum_{\substack{k=0 \\ k \neq q}}^{n-1} \mathbf{f}_{qk} = -Gm_q \sum_{\substack{k=0 \\ k \neq q}}^{n-1} \frac{m_k}{|\mathbf{s}_q(t) - \mathbf{s}_k(t)|^3} [\mathbf{s}_q(t) - \mathbf{s}_k(t)]. \quad (7.2)$$

Recall that the acceleration of an object is given by the second derivative of its position and that Newton's second law of motion states that the force on an object is given by its mass multiplied by its acceleration, so if the acceleration of particle q is $\mathbf{a}_q(t)$, then $\mathbf{F}_q(t) = m_q \mathbf{a}_q(t) = m_q \mathbf{s}_q''(t)$, where $\mathbf{s}_q''(t)$ is the second derivative of the position $\mathbf{s}_q(t)$. Thus, we can use Formula (7.2) to find the acceleration of particle q :

$$\mathbf{s}_q''(t) = -G \sum_{\substack{j=0 \\ j \neq q}}^{n-1} \frac{m_j}{|\mathbf{s}_q(t) - \mathbf{s}_j(t)|^3} [\mathbf{s}_q(t) - \mathbf{s}_j(t)]. \quad (7.3)$$

Thus Newton's laws give us a system of *differential* equations—equations involving derivatives—and our job is to find at each time t of interest the position $\mathbf{s}_q(t)$ and velocity $\mathbf{v}_q(t) = \mathbf{s}_q'(t)$.

We'll suppose that we either want to find the positions and velocities at the times

$$t = 0, \Delta t, 2\Delta t, \dots, T\Delta t,$$

or, more often, simply the positions and velocities at the final time $T\Delta t$. Here, Δt and T are specified by the user, so the input to the program will be n , the number of particles, Δt , T , and, for each particle, its mass, its initial position, and its initial velocity. In a fully general solver, the positions and velocities would be three-dimensional vectors, but to keep things simple, we'll assume that the particles will move in a plane, and we'll use two-dimensional vectors instead.

The output of the program will be the positions and velocities of the n particles at the timesteps $0, \Delta t, 2\Delta t, \dots$, or just the positions and velocities at $T\Delta t$. To get the

output at only the final time, we can add an input option in which the user specifies whether she only wants the final positions and velocities.

7.1.2 Two serial programs

In outline, a serial n -body solver can be based on the following pseudocode:

```

1   Get input data;
2   for each timestep {
3       if (timestep output)
4           Print positions and velocities of particles;
5       for each particle  $q$ 
6           Compute total force on  $q$ ;
7       for each particle  $q$ 
8           Compute position and velocity of  $q$ ;
9   }
10  Print positions and velocities of particles;
```

We can use our formula for the total force on a particle (Formula (7.2)) to refine our pseudocode for the computation of the forces in Lines 5–6:

```

for each particle  $q$  {
    for each particle  $k \neq q$  {
         $x\_diff = pos[q][X] - pos[k][X]$ ;
         $y\_diff = pos[q][Y] - pos[k][Y]$ ;
         $dist = \sqrt{x\_diff*x\_diff + y\_diff*y\_diff}$ ;
         $dist\_cubed = dist*dist*dist$ ;
         $forces[q][X] -= G*masses[q]*masses[k]/dist\_cubed * x\_diff$ ;
         $forces[q][Y] -= G*masses[q]*masses[k]/dist\_cubed * y\_diff$ ;
    }
}
```

Here, we’re assuming that the forces and the positions of the particles are stored as two-dimensional arrays, `forces` and `pos`, respectively. We’re also assuming we’ve defined constants $X = 0$ and $Y = 1$. So the x -component of the force on particle q is `forces[q][X]` and the y -component is `forces[q][Y]`. Similarly, the components of the position are `pos[q][X]` and `pos[q][Y]`. (We’ll take a closer look at data structures shortly.)

We can use Newton’s third law of motion, that is, for every action there is an equal and opposite reaction, to halve the total number of calculations required for the forces. If the force on particle q due to particle k is $\mathbf{f}_{qk}$, then the force on k due to q is $-\mathbf{f}_{qk}$. Using this simplification we can modify our code to compute forces, as shown in Program 7.1. To better understand this pseudocode, imagine the individual forces

```

for each particle q
    forces[q] = 0;
for each particle q {
    for each particle k > q {
        x_diff = pos[q][X] - pos[k][X];
        y_diff = pos[q][Y] - pos[k][Y];
        dist = sqrt(x_diff*x_diff + y_diff*y_diff);
        dist_cubed = dist*dist*dist;
        force_qk[X] = G*masses[q]*masses[k]/dist_cubed * x_diff;
        force_qk[Y] = G*masses[q]*masses[k]/dist_cubed * y_diff;

        forces[q][X] += force_qk[X];
        forces[q][Y] += force_qk[Y];
        forces[k][X] -= force_qk[X];
        forces[k][Y] -= force_qk[Y];
    }
}

```

Program 7.1: A reduced algorithm for computing n -body forces.

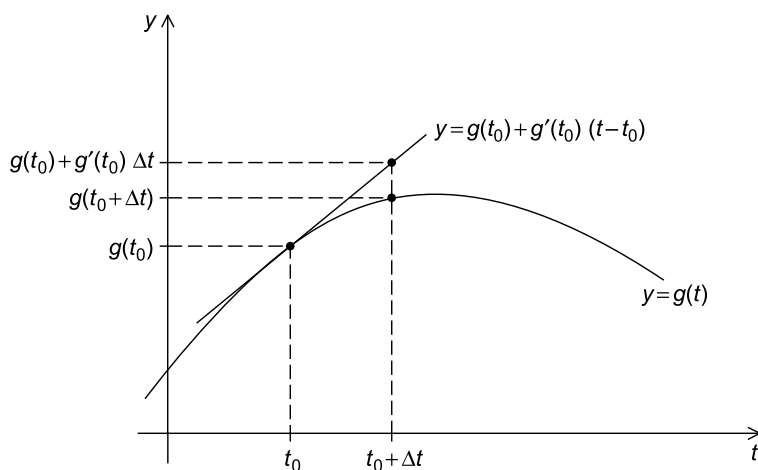
as a two-dimensional array:

$$\begin{bmatrix}
 0 & \mathbf{f}_{01} & \mathbf{f}_{02} & \cdots & \mathbf{f}_{0,n-1} \\
 -\mathbf{f}_{01} & 0 & \mathbf{f}_{12} & \cdots & \mathbf{f}_{1,n-1} \\
 -\mathbf{f}_{02} & -\mathbf{f}_{12} & 0 & \cdots & \mathbf{f}_{2,n-1} \\
 \vdots & \vdots & \vdots & \ddots & \vdots \\
 -\mathbf{f}_{0,n-1} & -\mathbf{f}_{1,n-1} & -\mathbf{f}_{2,n-1} & \cdots & 0
 \end{bmatrix}.$$

(Why are the diagonal entries 0?) Our original solver simply adds all of the entries in row q to get `forces[q]`. In our modified solver, when $q = 0$, the body of the loop **for** each particle q will add the entries in row 0 into `forces[0]`. It will also add the k th entry in column 0 into `forces[k]`, for $k = 1, 2, \dots, n - 1$. In general, the q th iteration will add the entries to the right of the diagonal (that is, to the right of the 0) in row q into `forces[q]`, and the entries below the diagonal in column q will be added into their respective forces, that is, the k th entry will be added into `forces[k]`.

Note that in using this modified solver, it's necessary to initialize the `forces` array in a separate loop, since the q th iteration of the loop that calculates the forces will, in general, add the values it computes into `forces[k]` for $k = q + 1, q + 2, \dots, n - 1$, not just `forces[q]`.

To distinguish between the two algorithms, we'll call the n -body solver with the original force calculation, the *basic* algorithm, and the solver with the number of calculations reduced, the *reduced* algorithm.

**FIGURE 7.1**

Using the tangent line to approximate a function.

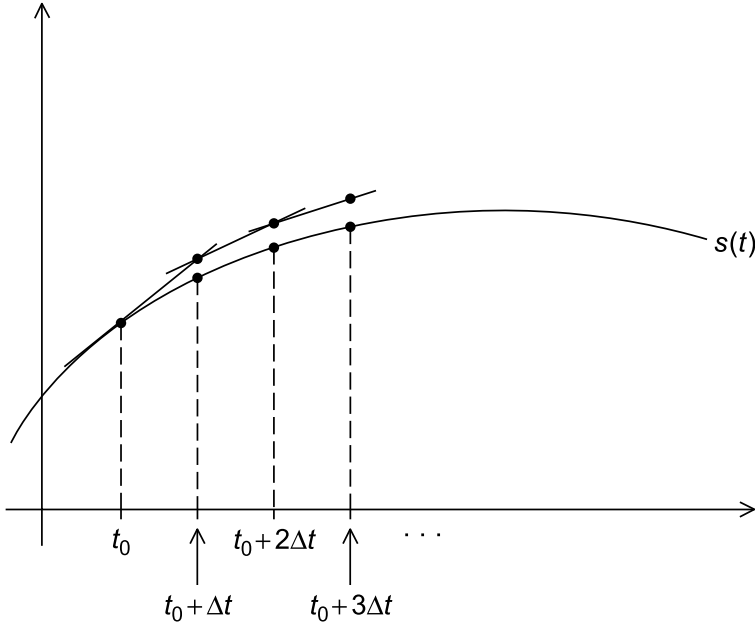
The position and the velocity remain to be found. We know that the acceleration of particle q is given by

$$\mathbf{a}_q(t) = \mathbf{s}_q''(t) = \mathbf{F}_q(t)/m_q,$$

where $\mathbf{s}_q''(t)$ is the second derivative of the position $\mathbf{s}_q(t)$ and $\mathbf{F}_q(t)$ is the force on particle q . We also know that the velocity $\mathbf{v}_q(t)$ is the first derivative of the position $\mathbf{s}_q(t)$, so we need to integrate the acceleration to get the velocity, and we need to integrate the velocity to get the position.

We might at first think that we can simply find an antiderivative of the function in Formula (7.3). However, a second look shows us that this approach has problems: the right-hand side contains unknown functions $\mathbf{s}_q$ and $\mathbf{s}_k$ —not just the variable t —so we'll instead use a **numerical** method for *estimating* the position and the velocity. This means that rather than trying to find simple closed formulas, we'll approximate the values of the position and velocity at the times of interest. There are *many* possible choices for numerical methods, but we'll use the simplest one: Euler's method, which is named after the famous Swiss mathematician Leonhard Euler (1707–1783). In Euler's method, we use the tangent line to approximate a function. The basic idea is that if we know the value of a function $g(t_0)$ at time t_0 and we also know its derivative $g'(t_0)$ at time t_0 , then we can approximate its value at time $t_0 + \Delta t$ by using the tangent line to the graph of $g(t_0)$. See Fig. 7.1 for an example. Now if we know a point $(t_0, g(t_0))$ on a line, and we know the slope of the line $g'(t_0)$, then an equation for the line is given by

$$y = g(t_0) + g'(t_0)(t - t_0).$$

**FIGURE 7.2**

Euler's method.

Since we're interested in the time $t = t_0 + \Delta t$, we get

$$g(t + \Delta t) \approx g(t_0) + g'(t_0)(t + \Delta t - t) = g(t_0) + \Delta t g'(t_0).$$

Note that this formula will work even when $g(t)$ and y are vectors: when this is the case, $g'(t)$ is also a vector and the formula just adds a vector to a vector multiplied by a scalar, Δt .

Now we know the value of $\mathbf{s}_q(t)$ and $\mathbf{s}'_q(t)$ at time 0, so we can use the tangent line and our formula for the acceleration to compute $\mathbf{s}_q(\Delta t)$ and $\mathbf{v}_q(\Delta t)$:

$$\mathbf{s}_q(\Delta t) \approx \mathbf{s}_q(0) + \Delta t \mathbf{s}'_q(0) = \mathbf{s}_q(0) + \Delta t \mathbf{v}_q(0),$$

$$\mathbf{v}_q(\Delta t) \approx \mathbf{v}_q(0) + \Delta t \mathbf{v}'_q(0) = \mathbf{v}_q(0) + \Delta t \mathbf{a}_q(0) = \mathbf{v}_q(0) + \Delta t \frac{1}{m_q} \mathbf{F}_q(0).$$

When we try to extend this approach to the computation of $\mathbf{s}_q(2\Delta t)$ and $\mathbf{s}'_q(2\Delta t)$, we see that things are a little bit different, since we don't know the exact value of $\mathbf{s}_q(\Delta t)$ and $\mathbf{s}'_q(\Delta t)$. However, if our approximations to $\mathbf{s}_q(\Delta t)$ and $\mathbf{s}'_q(\Delta t)$ are good, then we should be able to get a reasonably good approximation to $\mathbf{s}_q(2\Delta t)$ and $\mathbf{s}'_q(2\Delta t)$ using the same idea. This is what Euler's method does (see Fig. 7.2).

Now we can complete our pseudocode for the two n -body solvers by adding in the code for computing position and velocity:

```

pos[q][X] += delta_t*vel[q][X];
pos[q][Y] += delta_t*vel[q][Y];
vel[q][X] += delta_t/masses[q]*forces[q][X];
vel[q][Y] += delta_t/masses[q]*forces[q][Y];

```

Here, we're using `pos[q]`, `vel[q]`, and `forces[q]` to store the position, the velocity, and the force, respectively, of particle q .

Before moving on to parallelizing our serial program, let's take a moment to look at data structures. We've been using an array type to store our vectors:

```
#define DIM 2
```

```
typedef double vect_t[DIM];
```

A struct is also an option. However, if we're using arrays and we decide to change our program so that it solves three-dimensional problems, in principle we only need to change the macro `DIM`. If we try to do this with structs, we'll need to rewrite the code that accesses individual components of the vector.

For each particle, we need to know the values of

- its mass,
- its position,
- its velocity,
- its acceleration, and
- the total force acting on it.

Since we're using Newtonian physics, the mass of each particle is constant, but the other values will, in general, change as the program proceeds. If we examine our code, we'll see that once we've computed a new value for one of these variables for a given timestep, we never need the old value again. For example, we don't need to do anything like this:

```

new_pos_q = f(old_pos_q);
new_vel_q = g(old_pos_q, new_pos_q);

```

Also, the acceleration is only used to compute the velocity, and its value can be computed in one arithmetic operation from the total force, so we only need to use a local, temporary variable for the acceleration.

For each particle it suffices to store its mass and the current value of its position, velocity, and force. We could store these four variables as a struct and use an array of structs to store the data for all the particles. Of course, there's no reason that all of the variables associated with a particle need to be grouped together in a struct. We can split the data into separate arrays in a variety of different ways. We've chosen to group the mass, position, and velocity into a single struct and store the forces in a separate array. With the forces stored in contiguous memory, we can use a fast function, such as `memset`, to quickly assign zeroes to all of the elements at the beginning of each iteration:

```

#include <string.h>  /* For memset */
. . .
vect_t* forces = malloc(n*sizeof(vect_t));
. . .
for (step = 1; step <= n_steps; step++) {
    . . .
    /* Assign 0 to each element of the forces array */
    forces = memset(forces, 0, n*sizeof(vect_t));
    for (part = 0; part < n-1; part++)
        Compute_force(part, forces, . . .)
    . . .
}

```

If the force on each particle were a member of a struct, the force members wouldn't occupy contiguous memory in an array of structs, and we'd have to use a relatively slow **for** loop to assign zero to each element.

7.1.3 Parallelizing the n -body solvers

Let's try to apply Foster's methodology to the n -body solver. Since we initially want *lots* of tasks, we can start by making our tasks the computations of the positions, the velocities, and the total forces at each timestep. In the basic algorithm, the algorithm in which the total force on each particle is calculated directly from Formula (7.2), the computation of $\mathbf{F}_q(t)$, the total force on particle q at time t , requires the positions of each of the particles $\mathbf{s}_r(t)$, for each r . The computation of $\mathbf{v}_q(t + \Delta t)$ requires the velocity at the previous timestep, $\mathbf{v}_q(t)$, and the force, $\mathbf{F}_q(t)$, at the previous timestep. Finally, the computation of $\mathbf{s}_q(t + \Delta t)$ requires $\mathbf{s}_q(t)$ and $\mathbf{v}_q(t)$. The communications among the tasks can be illustrated as shown in Fig. 7.3. The figure makes it clear that most of the communication among the tasks occurs among the tasks associated with an individual particle, so if we agglomerate the computations of $\mathbf{s}_q(t)$, $\mathbf{v}_q(t)$,

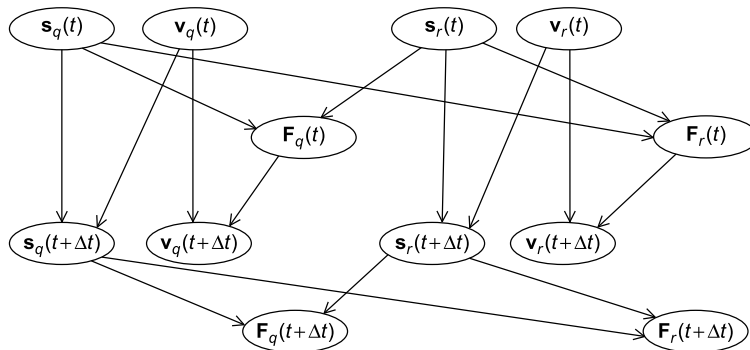
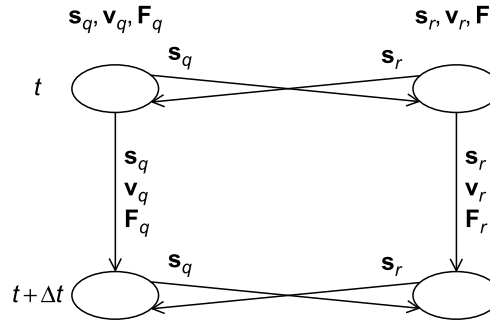
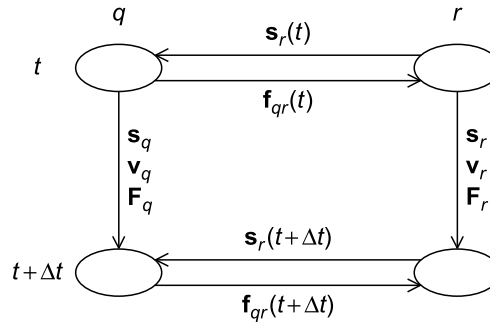


FIGURE 7.3

Communications among tasks in the basic n -body solver.

**FIGURE 7.4**

Communications among agglomerated tasks in the basic n -body solver.

**FIGURE 7.5**

Communications among agglomerated tasks in the reduced n -body solver ($q < r$).

and $\mathbf{F}_q(t)$, our intertask communication is greatly simplified (see Fig. 7.4). Now the (agglomerated) tasks correspond to the particles and, in the figure, we've labeled the communications with the data that's being communicated. For example, the arrow from particle q at timestep t to particle r at timestep t is labeled with $\mathbf{s}_q$, the position of particle q .

For the reduced algorithm, the “intra-particle” communications are the same. That is, to compute $\mathbf{s}_q(t + \Delta t)$, we'll need $\mathbf{s}_q(t)$ and $\mathbf{v}_q(t)$, and to compute $\mathbf{v}_q(t + \Delta t)$, we'll need $\mathbf{v}_q(t)$ and $\mathbf{F}_q(t)$. Therefore, once again it makes sense to agglomerate the computations associated with a single particle into a composite task.

Recall that in the reduced algorithm, we make use of the fact that the force $\mathbf{f}_{rq} = -\mathbf{f}_{qr}$. So if $q < r$, then the communication *from* task r *to* task q is the same as in the basic algorithm—to compute $\mathbf{F}_q(t)$, task/particle q will need $\mathbf{s}_r(t)$ from task/particle r . However, the communication from task q to task r is no longer $\mathbf{s}_q(t)$, it's the force on particle q due to particle r , that is, $\mathbf{f}_{qr}(t)$. (See Fig. 7.5.)

The final stage in Foster's methodology is mapping. If we have n particles and T timesteps, then there will be nT tasks in both the basic and the reduced algorithm.

Astrophysical n -body problems typically involve thousands or even millions of particles, so n may be several orders of magnitude greater than the number of available cores. However, T may also be much larger than the number of available cores. In principle, then, we have two “dimensions” to work with when we map tasks to cores. However, if we consider the nature of Euler’s method, we’ll see that attempting to assign tasks associated with a single particle at different timesteps to different cores won’t work very well. Before estimating $\mathbf{s}_q(t + \Delta t)$ and $\mathbf{v}_q(t + \Delta t)$, Euler’s method must “know” $\mathbf{s}_q(t)$, $\mathbf{v}_q(t)$, and $\mathbf{a}_q(t)$. Thus, if we assign particle q at time t to core c_0 , and we assign particle q at time $t + \Delta t$ to core $c_1 \neq c_0$, then we’ll have to communicate $\mathbf{s}_q(t)$, $\mathbf{v}_q(t)$, and $\mathbf{F}_q(t)$ from c_0 to c_1 . Of course, if particle q at time t and particle q at time $t + \Delta t$ are mapped to the same core, this communication won’t be necessary, so once we’ve mapped the task consisting of the calculations for particle q at the first timestep to core c_0 , we may as well map the subsequent computations for particle q to the same cores, since we can’t simultaneously execute the computations for particle q at two different timesteps. Thus, mapping tasks to cores will, in effect, be an assignment of particles to cores.

At first glance, it might seem that any assignment of particles to cores that assigns roughly $n/\text{thread_count}$ particles to each core will do a good job of balancing the workload among the cores, and for the basic algorithm this is the case. In the basic algorithm, the work required to compute the position, velocity, and force is the same for every particle. However, in the reduced algorithm, the work required in the forces computation loop is much greater for lower-numbered iterations than the work required for higher-numbered iterations. To see this, recall the pseudocode that computes the total force on particle q in the reduced algorithm:

```

for each particle  $k > q$  {
     $x\_diff = pos[q][X] - pos[k][X]$ ;
     $y\_diff = pos[q][Y] - pos[k][Y]$ ;
     $dist = \sqrt{x\_diff*x\_diff + y\_diff*y\_diff}$ ;
     $dist\_cubed = dist*dist*dist$ ;
     $force\_qk[X] = G*masses[q]*masses[k]/dist\_cubed * x\_diff$ ;
     $force\_qk[Y] = G*masses[q]*masses[k]/dist\_cubed * y\_diff$ ;

     $forces[q][X] += force\_qk[X]$ ;
     $forces[q][Y] += force\_qk[Y]$ ;
     $forces[k][X] -= force\_qk[X]$ ;
     $forces[k][Y] -= force\_qk[Y]$ ;
}

```

Then, for example, when $q = 0$, we’ll make $n - 1$ passes through the **for** each particle $k > q$ loop, while when $q = n - 1$, we won’t make any passes through the loop. Thus, for the reduced algorithm we would expect that a cyclic partition of the particles would do a better job than a block partition of evenly distributing the *computation*.

However, in a shared-memory setting, a cyclic partition of the particles among the cores is almost certain to result in a much higher number of cache misses than a block

partition, and in a distributed-memory setting, the overhead involved in communicating data that has a cyclic distribution will probably be greater than the overhead involved in communicating data that has a block distribution (see Exercises 7.8 and 7.10).

Therefore, with a composite task consisting of all of the computations associated with a single particle throughout the simulation, we conclude the following:

1. A block distribution will give the best performance for the basic n -body solver.
2. For the reduced n -body solver, a cyclic distribution will best distribute the workload in the computation of the forces. However, this improved performance *may* be offset by the cost of reduced cache performance in a shared-memory setting and additional communication overhead in a distributed-memory setting.

To make a final determination of the optimal mapping of tasks to cores, we'll need to do some experimentation.

7.1.4 A word about I/O

You may have noticed that our discussion of parallelizing the n -body solver hasn't touched on the issue of I/O, even though I/O can figure prominently in both of our serial algorithms. We've discussed the problem of I/O several times in earlier chapters. Recall that different parallel systems vary widely in their I/O capabilities, and with the very basic I/O that is commonly available it is very difficult to obtain high performance. This basic I/O was designed for use by single-process, single-threaded programs, and when multiple processes or multiple threads attempt to access the I/O buffers, the system makes no attempt to schedule their access. For example, if multiple threads attempt to execute

```
printf("Hello from thread %d of %d\n", my_rank, thread_count);
```

more or less simultaneously, the order in which the output appears will be unpredictable. Even worse, one thread's output may not even appear as a single line. It can happen that the output from one thread appears as multiple segments, and the individual segments are separated by output from other threads.

Thus, as we've noted earlier, except for debug output, we generally assume that one process/thread does all the I/O, and when we're timing program execution, we'll use the option to only print output for the final timestep. Furthermore, we won't include this output in the reported run-times.

Of course, even if we're ignoring the cost of I/O, we can't ignore its existence. We'll briefly discuss its implementation when we discuss the details of our parallel implementations.

7.1.5 Parallelizing the basic solver using OpenMP

How can we use OpenMP to map tasks/particles to cores in the basic version of our n -body solver? Let's take a look at the pseudocode for the serial program:

```

for each timestep {
    if (timestep output)
        Print positions and velocities of particles;
    for each particle q
        Compute total force on q;
    for each particle q
        Compute position and velocity of q;
}

```

The two inner loops are both iterating over particles. So, in principle, parallelizing the two inner **for** loops will map tasks/particles to cores, and we might try something like this:

```

    for each timestep {
        if (timestep output)
            Print positions and velocities of particles;
#    pragma omp parallel for
        for each particle q
            Compute total force on q;
#    pragma omp parallel for
        for each particle q
            Compute position and velocity of q;
    }

```

We may not like the fact that this code could do a lot of forking and joining of threads, but before dealing with that, let's take a look at the loops themselves: we need to see if there are any race conditions caused by loop-carried dependences.

In the basic version, the first loop has the following form:

```

# pragma omp parallel for
for each particle q {
    forces[q][X] = forces[q][Y] = 0;
    for each particle k != q {
        x_diff = pos[q][X] - pos[k][X];
        y_diff = pos[q][Y] - pos[k][Y];
        dist = sqrt(x_diff*x_diff + y_diff*y_diff);
        dist_cubed = dist*dist*dist;
        forces[q][X]
            -= G*masses[q]*masses[k]/dist_cubed*x_diff;
        forces[q][Y]
            -= G*masses[q]*masses[k]/dist_cubed*y_diff;
    }
}

```

Since the iterations of the **for** each particle *q* loop are partitioned among the threads, only one thread will access `forces[q]` for any *q*. Different threads do access the same elements of the `pos` array and the `masses` array. However, these arrays are only *read* in the loop. The remaining variables are used for temporary storage in a single iteration of the inner loop, and they can be private. Thus, the parallelization of the first loop in the basic algorithm won't introduce any race conditions.

The second loop has the form:

```
# pragma omp parallel for
for each particle q {
    pos[q][X] += delta_t*vel[q][X];
    pos[q][Y] += delta_t*vel[q][Y];
    vel[q][X] += delta_t/masses[q]*forces[q][X];
    vel[q][Y] += delta_t/masses[q]*forces[q][Y];
}
```

Here, a single thread accesses `pos[q]`, `vel[q]`, `masses[q]`, and `forces[q]`, for any particle q , and the scalar variables are only read, so parallelizing this loop also won't introduce any race conditions.

Let's return to the issue of repeated forking and joining of threads. In our pseudocode, we have

```
for each timestep {
    if (timestep output)
        Print positions and velocities of particles;
#    pragma omp parallel for
    for each particle q
        Compute total force on q;
#    pragma omp parallel for
    for each particle q
        Compute position and velocity of q;
}
```

We encountered a similar issue when we parallelized odd-even transposition sort (see Section 5.6.2). In that case, we put a `parallel` directive before the outermost loop and used OpenMP `for` directives for the inner loops. Will a similar strategy work here? That is, can we do something like this?

```
# pragma omp parallel
for each timestep {
    if (timestep output)
        Print positions and velocities of particles;
#    pragma omp for
    for each particle q
        Compute total force on q;
#    pragma omp for
    for each particle q
        Compute position and velocity of q;
}
```

This will have the desired effect on the two `for each particle` loops: the same team of threads will be used in both loops and for every iteration of the outer loop. However, we have a clear problem with the output statement. As it stands now, every thread will print all the positions and velocities, and we only want one thread to do the I/O. However, OpenMP provides the `single` directive for exactly this situation: we have a team of threads executing a block of code, but a part of the code should only be

executed by one of the threads. Adding the `single` directive gives us the following pseudocode:

```
# pragma omp parallel
  for each timestep {
    if (timestep output) {
#      pragma omp single
        Print positions and velocities of particles;
    }
#    pragma omp for
    for each particle q
        Compute total force on q;
#    pragma omp for
    for each particle q
        Compute position and velocity of q;
  }
```

There are still a few issues that we need to address. The most important has to do with possible race conditions introduced in the transition from one statement to another. For example, suppose thread 0 completes the first `for each particle` loop before thread 1, and it then starts updating the positions and velocities of its assigned particles in the second `for each particle` loop. Clearly, this could cause thread 1 to use an updated position in the first `for each particle` loop. However, recall that there is an implicit barrier at the end of each structured block that has been parallelized with a `for` directive. So, if thread 0 finishes the first inner loop before thread 1, it will block until thread 1 (and any other threads) finish the first inner loop, and it won't start the second inner loop until all the threads have finished the first. This will also prevent the possibility that a thread might rush ahead and print positions and velocities before they've all been updated by the second loop.

There's also an implicit barrier after the `single` directive, although in this program the barrier isn't necessary. Since the output statement won't update any memory locations, it's OK for some threads to go ahead and start executing the next iteration before output has been completed. Furthermore, the first inner `for` loop in the next iteration only updates the `forces` array, so it can't cause a thread executing the output statement to print incorrect values, and because of the barrier at the end of the first inner loop, no thread can race ahead and start updating positions and velocities in the second inner loop before the output has been completed. Thus, we could modify the `single` directive with a `nowait` clause. If the OpenMP implementation supports it, this simply eliminates the implied barrier associated with the `single` directive. It can also be used with `for`, `parallel for`, and `parallel` directives. Note that in this case, addition of the `nowait` clause is unlikely to have much effect on performance, since the two `for each particle` loops have implied barriers that will prevent any one thread from getting more than a few statements ahead of any other.

Finally, we may want to add a schedule clause to each of the `for` directives to ensure that the iterations have a block partition:

```
#    pragma omp for schedule(static, n/thread_count)
```

7.1.6 Parallelizing the reduced solver using OpenMP

The reduced solver has an additional inner loop: the initialization of the `forces` array to 0. If we try to use the same parallelization for the reduced solver, we should also parallelize this loop with a `for` directive. What happens if we try this? That is, what happens if we try to parallelize the reduced solver with the following pseudocode?

```
# pragma omp parallel
  for each timestep {
    if (timestep output) {
#      pragma omp single
        Print positions and velocities of particles;
    }
#    pragma omp for
    for each particle q
        forces[q] = 0.0;
#    pragma omp for
    for each particle q
        Compute total force on q;
#    pragma omp for
    for each particle q
        Compute position and velocity of q;
  }
```

Parallelization of the initialization of the forces should be fine, since there's no dependence among the iterations. The updating of the positions and velocities is the same in both the basic and reduced solvers, so if the computation of the forces is OK, then this should also be OK.

How does parallelization affect the correctness of the loop for computing the forces? Recall that in the reduced version, this loop has the following form:

```
# pragma omp for /* Can be faster than memset */
  for each particle q {
    force_qk[X] = force_qk[Y] = 0;
    for each particle k > q {
      x_diff = pos[q][X] - pos[k][X];
      y_diff = pos[q][Y] - pos[k][Y];
      dist = sqrt(x_diff*x_diff + y_diff*y_diff);
      dist_cubed = dist*dist*dist;
      force_qk[X]
        = G*masses[q]*masses[k]/dist_cubed * x_diff;
      force_qk[Y]
        = G*masses[q]*masses[k]/dist_cubed * y_diff;

      forces[q][X] += force_qk[X];
      forces[q][Y] += force_qk[Y];
      forces[k][X] -= force_qk[X];
      forces[k][Y] -= force_qk[Y];
    }
  }
```

As before, the variables of interest are `pos`, `masses`, and `forces`, since the values in the remaining variables are only used in a single iteration, and hence can be private. Also, as before, elements of the `pos` and `masses` arrays are only read, not updated. We therefore need to look at the elements of the `forces` array. In this version, unlike the basic version, a thread *may* update elements of the `forces` array other than those corresponding to its assigned particles. For example, suppose we have two threads and four particles and we're using a block partition of the particles. Then the total force on particle 3 is given by

$$\mathbf{F}_3 = -\mathbf{f}_{03} - \mathbf{f}_{13} - \mathbf{f}_{23}.$$

Furthermore, thread 0 will compute $\mathbf{f}_{03}$ and $\mathbf{f}_{13}$, while thread 1 will compute $\mathbf{f}_{23}$. Thus, the updates to `forces[3]` *do* create a race condition. In general, then, the updates to the elements of the `forces` array introduce race conditions into the code.

A seemingly obvious solution to this problem is to use a `critical` directive to limit access to the elements of the `forces` array. There are at least a couple of ways to do this. The simplest is to put a `critical` directive before all the updates to `forces`:

```
# pragma omp critical
{
    forces[q][X] += force_qk[X];
    forces[q][Y] += force_qk[Y];
    forces[k][X] -= force_qk[X];
    forces[k][Y] -= force_qk[Y];
}
```

However, with this approach access to the elements of the `forces` array will be effectively serialized. Only one element of `forces` can be updated at a time, and contention for access to the critical section is actually likely to seriously degrade the performance of the program. (See Exercise 7.3.)

An alternative would be to have one critical section for each particle. However, as we've seen, OpenMP doesn't readily support varying numbers of critical sections, so we would need to use one lock for each particle instead and our updates would look something like this:

```
omp_set_lock(&locks[q]);
forces[q][X] += force_qk[X];
forces[q][Y] += force_qk[Y];
omp_unset_lock(&locks[q]);

omp_set_lock(&locks[k]);
forces[k][X] -= force_qk[X];
forces[k][Y] -= force_qk[Y];
omp_unset_lock(&locks[k]);
```

This assumes that the master thread will create a shared array of locks, one for each particle, and when we update an element of the `forces` array, we first set the lock

Table 7.1 First-phase computations for reduced algorithm with block partition.

Thread	Particle	Thread		
		0	1	2
0	0	$\mathbf{f}_{01} + \mathbf{f}_{02} + \mathbf{f}_{03} + \mathbf{f}_{04} + \mathbf{f}_{05}$	0	0
	1	$-\mathbf{f}_{01} + \mathbf{f}_{12} + \mathbf{f}_{13} + \mathbf{f}_{14} + \mathbf{f}_{15}$	0	0
1	2	$-\mathbf{f}_{02} - \mathbf{f}_{12}$	$\mathbf{f}_{23} + \mathbf{f}_{24} + \mathbf{f}_{25}$	0
	3	$-\mathbf{f}_{03} - \mathbf{f}_{13}$	$-\mathbf{f}_{23} + \mathbf{f}_{34} + \mathbf{f}_{35}$	0
2	4	$-\mathbf{f}_{04} - \mathbf{f}_{14}$	$-\mathbf{f}_{24} - \mathbf{f}_{34}$	$\mathbf{f}_{45}$
	5	$-\mathbf{f}_{05} - \mathbf{f}_{15}$	$-\mathbf{f}_{25} - \mathbf{f}_{35}$	$-\mathbf{f}_{45}$

Table 7.2 First-phase computations for reduced algorithm with cyclic partition.

Thread	Particle	Thread		
		0	1	2
0	0	$\mathbf{f}_{01} + \mathbf{f}_{02} + \mathbf{f}_{03} + \mathbf{f}_{04} + \mathbf{f}_{05}$	0	0
1	1	$-\mathbf{f}_{01}$	$\mathbf{f}_{12} + \mathbf{f}_{13} + \mathbf{f}_{14} + \mathbf{f}_{15}$	0
2	2	$-\mathbf{f}_{02}$	$-\mathbf{f}_{12}$	$\mathbf{f}_{23} + \mathbf{f}_{24} + \mathbf{f}_{25}$
0	3	$-\mathbf{f}_{03} + \mathbf{f}_{34} + \mathbf{f}_{35}$	$-\mathbf{f}_{13}$	$-\mathbf{f}_{23}$
1	4	$-\mathbf{f}_{04} - \mathbf{f}_{34}$	$-\mathbf{f}_{14} + \mathbf{f}_{45}$	$-\mathbf{f}_{24}$
2	5	$-\mathbf{f}_{05} - \mathbf{f}_{35}$	$-\mathbf{f}_{15} - \mathbf{f}_{45}$	$-\mathbf{f}_{25}$

corresponding to that particle. Although this approach performs much better than the single critical section, it still isn't competitive with the serial code. (See Exercise 7.4.)

Another possible solution is to carry out the computation of the forces in two phases. In the first phase, each thread carries out exactly the same calculations it carried out in the erroneous parallelization. However, now the calculations are stored in its *own* array of forces. Then, in the second phase, the thread that has been assigned particle q will add the contributions that have been computed by the different threads. In our example above, thread 0 would compute $-\mathbf{f}_{03} - \mathbf{f}_{13}$, while thread 1 would compute $-\mathbf{f}_{23}$. After each thread was done computing its contributions to the forces, thread 1, which has been assigned particle 3, would find the total force on particle 3 by adding these two values.

Let's look at a slightly larger example. Suppose we have three threads and six particles. If we're using a block partition of the particles, then the computations in the first phase are shown in Table 7.1. The last three columns of the table show each thread's contribution to the computation of the total forces. In phase 2 of the computation, the thread specified in the first column of the table will add the contents of each of its assigned rows—that is, each of its assigned particles.

Note that there's nothing special about using a block partition of the particles. Table 7.2 shows the same computations if we use a cyclic partition of the particles. Note that if we compare this table with the table that shows the block partition, it's clear that the cyclic partition does a better job of balancing the load.

To implement this, during the first phase our revised algorithm proceeds as before, except that each thread adds the forces it computes into its own subarray of `loc_forces`:

```
# pragma omp for
for each particle q {
    force_qk[X] = force_qk[Y] = 0;
    for each particle k > q {
        x_diff = pos[q][X] - pos[k][X];
        y_diff = pos[q][Y] - pos[k][Y];
        dist = sqrt(x_diff*x_diff + y_diff*y_diff);
        dist_cubed = dist*dist*dist;
        force_qk[X]
            = G*masses[q]*masses[k]/dist_cubed * x_diff;
        force_qk[Y]
            = G*masses[q]*masses[k]/dist_cubed * y_diff;

        loc_forces[my_rank][q][X] += force_qk[X];
        loc_forces[my_rank][q][Y] += force_qk[Y];
        loc_forces[my_rank][k][X] -= force_qk[X];
        loc_forces[my_rank][k][Y] -= force_qk[Y];
    }
}
```

During the second phase, each thread adds the forces computed by all the threads for its assigned particles:

```
# pragma omp for
for (q = 0; q < n; q++) {
    forces[q][X] = forces[q][Y] = 0;
    for (thread = 0; thread < thread_count; thread++) {
        forces[q][X] += loc_forces[thread][q][X];
        forces[q][Y] += loc_forces[thread][q][Y];
    }
}
```

Before moving on, we should make sure that we haven't inadvertently introduced any new race conditions. During the first phase, since each thread writes to its own subarray, there isn't a race condition in the updates to `loc_forces`. Also, during the second phase, only the "owner" of thread q writes to `forces[q]`, so there are no race conditions in the second phase. Finally, since there is an implied barrier after each of the parallelized `for` loops, we don't need to worry that some thread is going to race ahead and make use of a variable that hasn't been properly initialized, or that some slow thread is going to make use of a variable that has had its value changed by another thread.

Table 7.3 Run-times of the n -body solvers parallelized with OpenMP (times in seconds).

Threads	Basic	Reduced Default Sched	Reduced Forces Cyclic	Reduced All Cyclic
1	7.71	3.90	3.90	3.90
2	3.87	2.94	1.98	2.01
4	1.95	1.73	1.01	1.08
8	0.99	0.95	0.54	0.61

7.1.7 Evaluating the OpenMP codes

Before we can compare the basic and the reduced codes, we need to decide how to schedule the parallelized **for** loops. For the basic code, we’ve seen that any schedule that divides the iterations equally among the threads should do a good job of balancing the computational load. (As usual, we’re assuming no more than one thread/core.) We also observed that a block partitioning of the iterations would result in fewer cache misses than a cyclic partition. Thus, we would expect that a block schedule would be the best option for the basic version.

In the reduced code, the amount of work done in the first phase of the computation of the forces decreases as the **for** loop proceeds. We’ve seen that a cyclic schedule should do a better job of assigning more or less equal amounts of work to each thread. In the remaining parallel **for** loops—the initialization of the `loc_forces` array, the second phase of the computation of the forces, and the updating of the positions and velocities—the work required is roughly the same for all the iterations. Therefore, *taken out of context*, each of these loops will probably perform best with a block schedule. However, the schedule of one loop can affect the performance of another (see Exercise 7.11), so it may be that choosing a cyclic schedule for one loop and block schedules for the others will degrade performance.

With these choices, Table 7.3 shows the performance of the n -body solvers when they’re run on one of our systems with no I/O. The solver used 400 particles for 1000 timesteps. The column labeled “Default Sched” gives times for the OpenMP reduced solver when all of the inner loops use the default schedule, which, on our system, is a block schedule. The column labeled “Forces Cyclic” gives times when the first phase of the forces computation uses a cyclic schedule and the other inner loops use the default schedule. The last column, labeled “All Cyclic,” gives times when all of the inner loops use a cyclic schedule. The run-times of the serial solvers differ from those of the single-threaded solvers by less than 1%, so we’ve omitted them from the table.

Notice that with more than one thread, the reduced solver, using all default schedules, takes anywhere from 50 to 75% longer than the reduced solver with the cyclic forces computation. Using the cyclic schedule is clearly superior to the default schedule in this case, and any loss in time resulting from cache issues is more than made up for by the improved load balance for the computations.

For only two threads, there is very little difference between the performance of the reduced solver with only the first forces loop cyclic and the reduced solver with all loops cyclic. However, as we increase the number of threads, the performance of the reduced solver that uses a cyclic schedule for all of the loops does start to degrade. In this particular case, when there are more threads, it appears that the overhead involved in changing distributions is less than the overhead incurred from false sharing.

Finally, notice that the basic solver takes about twice as long as the reduced solver with the cyclic scheduling of the forces computation. So if the extra memory is available, the reduced solver is clearly superior. However, the reduced solver increases the memory requirement for the storage of the forces by a factor of `thread_count`, so for very large numbers of particles, it may be impossible to use the reduced solver.

7.1.8 Parallelizing the solvers using Pthreads

Parallelizing the two n -body solvers using Pthreads is very similar to parallelizing them using OpenMP. The differences are only in implementation details, so rather than repeating the discussion, we will point out some of the principal differences between the Pthreads and the OpenMP implementations. We will also note some of the more important similarities.

- By default, local variables in Pthreads are private, so all shared variables are global in the Pthreads version.
- The principal data structures in the Pthreads version are identical to those in the OpenMP version: vectors are two-dimensional arrays of **doubles**, and the mass, position, and velocity of a single particle are stored in a struct. The forces are stored in an array of vectors.
- Startup for Pthreads is basically the same as the startup for OpenMP: the main thread gets the command-line arguments, and allocates and initializes the principal data structures.
- The main difference between the Pthreads and the OpenMP implementations is in the details of parallelizing the inner loops. Since Pthreads has nothing analogous to a `parallel for` directive, we must explicitly determine which values of the loop variables correspond to each thread's calculations. To facilitate this, we've written a function `Loop_schedule`, which determines
 - the initial value of the loop variable,
 - the final value of the loop variable, and
 - the increment for the loop variable.

The input to the function is

- the calling thread's rank,
 - the number of threads,
 - the total number of iterations, and
 - an argument indicating whether the partitioning should be block or cyclic.
- Another difference between the Pthreads and the OpenMP versions has to do with barriers. Recall that the end of a `parallel for` directive OpenMP has an implied barrier. As we've seen, this is important. For example, we don't want a thread

to start updating its positions until all the forces have been calculated, because it could use an out-of-date force and another thread could use an out-of-date position. If we simply partition the loop iterations among the threads in the Pthreads versions, there won't be a barrier at the end of an inner **for** loop and we'll have a race condition. Thus, we need to add explicit barriers after the inner loops when a race condition can arise. The Pthreads standard includes a barrier. However, some systems don't implement it, so we've defined a function that uses a Pthreads condition variable to implement a barrier. See Subsection 4.8.3 for details.

7.1.9 Parallelizing the basic solver using MPI

With our composite tasks corresponding to the individual particles, it's fairly straightforward to parallelize the basic algorithm using MPI. The only communication among the tasks occurs when we're computing the forces, and, to compute the forces, each task/particle needs the position and mass of every other particle. `MPI_Allgather` is expressly designed for this situation, since it collects on each process the same information from every other process. We've already noted that a block distribution will probably have the best performance, so we should use a block mapping of the particles to the processes.

In the shared-memory MIMD implementations, we collected most of the data associated with a single particle (mass, position, and velocity) into a single struct. However, if we use this data structure in the MPI implementation, we'll need to use a derived datatype in the call to `MPI_Allgather`, and communications with derived datatypes tend to be slower than communications with basic MPI types. Thus, it will make more sense to use individual arrays for the masses, positions, and velocities. We'll also need an array for storing the positions of all the particles. If each process has sufficient memory, then each of these can be a separate array. In fact, if memory isn't a problem, each process can store the entire array of masses, since these will never be updated and their values only need to be communicated during the initial setup.

On the other hand, if memory is short, there is an "in-place" option that can be used with some MPI collective communications. For our situation, suppose that the array `pos` can store the positions of all n particles. Further suppose that `vect_mpi_t` is an MPI datatype that stores two contiguous **doubles**. Also suppose that n is evenly divisible by `comm_sz` and `loc_n = n/comm_sz`. Then, if we store the local positions in a separate array, `loc_pos`, we can use the following call to collect all of the positions on each process:

```
MPI_Allgather(loc_pos, loc_n, vect_mpi_t,
              pos, loc_n, vect_mpi_t, comm);
```

If we can't afford the extra storage for `loc_pos`, then we can have each process q store its local positions in the q th block of `pos`. That is, the local positions of each process P should be stored in the appropriate block of each process' `pos` array:

```

P0: pos[0], pos[1], ... , pos[loc_n-1]
P1: pos[loc_n], pos[loc_n+1], ... , pos[loc_n + loc_n-1]
...
Pq: pos[q*loc_n], pos[q*loc_n+1], ... , pos[q*loc_n + loc_n-1]
...

```

With the `pos` array initialized this way on each process, we can use the following call to `MPI_Allgather`:

```

MPI_Allgather(MPI_IN_PLACE, loc_n, vect_mpi_t,
              pos, loc_n, vect_mpi_t, comm);

```

In this call, the first `loc_n` and `vect_mpi_t` arguments are ignored. However, it's not a bad idea to use arguments whose values correspond to the values that will be used, just to increase the readability of the program.

In the program we've written, we made the following choices with respect to the data structures:

- Each process stores the entire global array of particle masses.
- Each process only uses a single n -element array for the positions.
- Each process uses a pointer `loc_pos` that refers to the start of its block of `pos`. Thus, on process 0 `local_pos = pos`, on process 1 `local_pos = pos + loc_n`, and, so on.

With these choices, we can implement the basic algorithm with the pseudocode shown in Program 7.2. As usual, process 0 will read and broadcast the command

```

1 Get input data;
2 for each timestep {
3     if (timestep output)
4         Print positions and velocities of particles;
5     for each local particle loc_q
6         Compute total force on loc_q;
7     for each local particle loc_q
8         Compute position and velocity of loc_q;
9     Allgather local positions into global pos array;
10 }
11 Print positions and velocities of particles;

```

Program 7.2: Pseudocode for the MPI version of the basic n -body solver.

line arguments. It will also read the input and print the results. In Line 1, it will need to distribute the input data. Therefore, `Get input data` might be implemented as follows:

```

if (my_rank == 0) {
    for each particle
        Read masses[particle], pos[particle], vel[particle];
}

```

```

MPI_Bcast(masses, n, MPI_DOUBLE, 0, comm);
MPI_Bcast(pos, n, vect_mpi_t, 0, comm);
MPI_Scatter(vel, loc_n, vect_mpi_t,
            loc_vel, loc_n, vect_mpi_t,
            0, comm);

```

So process 0 reads all the initial conditions into three n -element arrays. Since we’re storing all the masses on each process, we broadcast `masses`. Also, since each process will need the global array of positions for the first computation of forces in the main **for** loop, we just broadcast `pos`. However, velocities are only used locally for the updates to positions and velocities, so we scatter `vel`.

Notice that we gather the updated positions in Line 9 at the end of the body of the outer **for** loop of Program 7.2. This ensures that the positions will be available for output in both Line 4 and Line 11. If we’re printing the results for each timestep, this placement allows us to eliminate an expensive collective communication call: if we simply gathered the positions onto process 0 before output, we’d have to call `MPI_Allgather` before the computation of the forces. With this organization of the body of the outer **for** loop, we can implement the output with the following pseudocode:

```

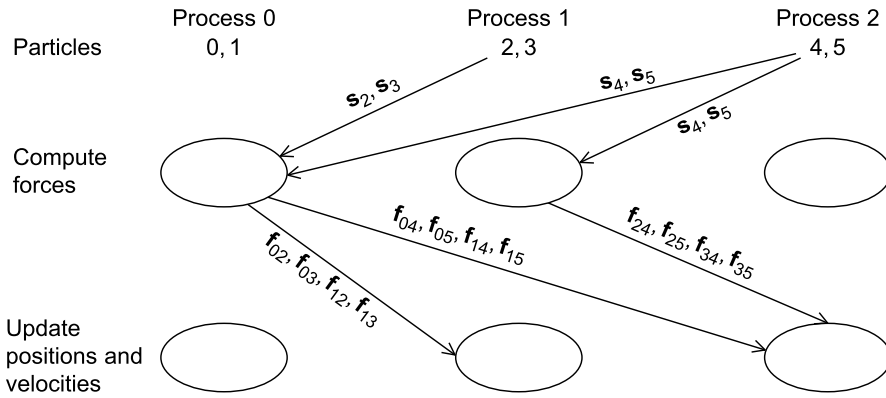
Gather velocities onto process 0;
if (my_rank == 0) {
    Print timestep;
    for each particle
        Print pos[particle] and vel[particle]
}

```

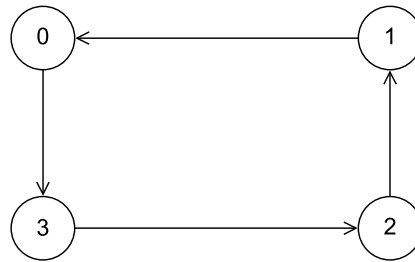
7.1.10 Parallelizing the reduced solver using MPI

The “obvious” implementation of the reduced algorithm is likely to be extremely complicated. Before computing the forces, each process will need to gather a subset of the positions, and after the computation of the forces, each process will need to scatter some of the individual forces it has computed and add the forces it receives. Fig. 7.6 shows the communications that would take place if we had three processes, six particles, and used a block partitioning of the particles among the processes. Not surprisingly, the communications are even more complex when we use a cyclic distribution (see Exercise 7.14). Certainly it would be possible to implement these communications. However, unless the implementation were *very* carefully done, it would probably be *very* slow.

Fortunately, there’s a much simpler alternative that uses a communication structure that is sometimes called a *ring pass*. In a ring pass, we imagine the processes as being interconnected in a ring (see Fig. 7.7). Process 0 communicates directly with processes 1 and `comm_sz - 1`, process 1 communicates with processes 0 and 2, and so on. The communication in a ring pass takes place in phases, and during each phase each process sends data to its “lower-ranked” neighbor, and receives data from its “higher-ranked” neighbor. Thus, 0 will send to `comm_sz - 1` and receive from 1.

**FIGURE 7.6**

Communication in a possible MPI implementation of the reduced n -body solver.

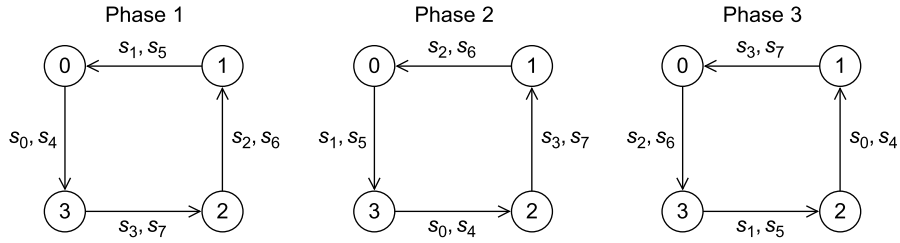
**FIGURE 7.7**

A ring of processes.

1 will send to 0 and receive from 2, and so on. In general, process q will send to process $(q - 1 + \text{comm_sz}) \% \text{comm_sz}$ and receive from process $(q + 1) \% \text{comm_sz}$.

By repeatedly sending and receiving data using this ring structure, we can arrange that each process has access to the positions of all the particles. During the first phase, each process will send the positions of its assigned particles to its “lower-ranked” neighbor and receive the positions of the particles assigned to its higher-ranked neighbor. During the next phase, each process will forward the positions it received in the first phase. This process continues through $\text{comm_sz} - 1$ phases until each process has received the positions of all of the particles. Fig. 7.8 shows the three phases if there are four processes and eight particles that have been cyclically distributed.

Of course, the virtue of the reduced algorithm is that we don’t need to compute all of the inter-particle forces, since $\mathbf{f}_{kq} = -\mathbf{f}_{qk}$ for every pair of particles q and k . To see how to exploit this, first observe that using the reduced algorithm, the interparticle forces can be divided into those that are *added* into and those that are subtracted from the total forces on the particle. For example, if we have six particles, then the

**FIGURE 7.8**

Ring pass of positions.

reduced algorithm will compute the force on particle 3 as

$$\mathbf{F}_3 = -\mathbf{f}_{03} - \mathbf{f}_{13} - \mathbf{f}_{23} + \mathbf{f}_{34} + \mathbf{f}_{35}.$$

The key to understanding the ring pass computation of the forces is to observe that the interparticle forces that are *subtracted* are computed by another task/particle, while the forces that are *added* are computed by the owning task/particle. Thus, the computations of the interparticle forces on particle 3 are assigned as follows:

Force	$\mathbf{f}_{03}$	$\mathbf{f}_{13}$	$\mathbf{f}_{23}$	$\mathbf{f}_{34}$	$\mathbf{f}_{35}$
Task/Particle	0	1	2	3	3

So, suppose that for our ring pass, instead of simply passing $\text{loc_n} = n/\text{comm_sz}$ positions, we also pass loc_n forces. Then in each phase, a process can

1. compute interparticle forces resulting from interaction between its assigned particles and the particles whose positions it has received, and
2. once an interparticle force has been computed, the process can add the force into a local array of forces corresponding to its particles, *and* it can subtract the interparticle force from the received array of forces.

See, for example, [17,37] for further details and alternatives.

Let's take a look at how the computation would proceed when we have four particles, two processes, and we're using a cyclic distribution of the particles among the processes (see Table 7.4). We're calling the arrays that store the local positions and local forces loc_pos and loc_forces , respectively. These are not communicated among the processes. The arrays that are communicated among the processes are tmp_pos and tmp_forces .

Before the ring pass can begin, both arrays storing positions are initialized with the positions of the local particles, and the arrays storing the forces are set to 0. Before the ring pass begins, each process computes those forces that are due to interaction among its assigned particles. Process 0 computes $\mathbf{f}_{02}$ and process 1 computes $\mathbf{f}_{13}$. These values are added into the appropriate locations in loc_forces and subtracted from the appropriate locations in tmp_forces .

Table 7.4 Computation of forces in ring pass.

Time	Variable	Process 0	Process 1
Start	loc_pos	s_0, s_2	s_1, s_3
	loc_forces	0, 0	0, 0
	tmp_pos	s_0, s_2	s_1, s_3
	tmp_forces	0, 0	0, 0
After Comp of Forces	loc_pos	s_0, s_2	s_1, s_3
	loc_forces	$f_{02}, 0$	$f_{13}, 0$
	tmp_pos	s_0, s_2	s_1, s_3
	tmp_forces	0, $-f_{02}$	0, $-f_{13}$
After First Comm	loc_pos	s_0, s_2	s_1, s_3
	loc_forces	$f_{02}, 0$	$f_{13}, 0$
	tmp_pos	s_1, s_3	s_0, s_2
	tmp_forces	0, $-f_{13}$	0, $-f_{02}$
After Comp of Forces	loc_pos	s_0, s_2	s_1, s_3
	loc_forces	$f_{01} + f_{02} + f_{03}, f_{23}$	$f_{12} + f_{13}, 0$
	tmp_pos	s_1, s_3	s_0, s_2
	tmp_forces	$-f_{01}, -f_{03} - f_{13} - f_{23}$	0, $-f_{02} - f_{12}$
After Second Comm	loc_pos	s_0, s_2	s_1, s_3
	loc_forces	$f_{01} + f_{02} + f_{03}, f_{23}$	$f_{12} + f_{13}, 0$
	tmp_pos	s_0, s_2	s_1, s_3
	tmp_forces	0, $-f_{02} - f_{12}$	$-f_{01}, -f_{03} - f_{13} - f_{23}$
After Comp of Forces	loc_pos	s_0, s_2	s_1, s_3
	loc_forces	$f_{01} + f_{02} + f_{03}, -f_{02} - f_{12} + f_{23}$	$-f_{01} + f_{12} + f_{13}, -f_{03} - f_{13} - f_{23}$
	tmp_pos	s_0, s_2	s_1, s_3
	tmp_forces	0, $-f_{02} - f_{12}$	$-f_{01}, -f_{03} - f_{13} - f_{23}$

Now, the two processes exchange `tmp_pos` and `tmp_forces` and compute the forces due to interaction among their local particles and the received particles. In the reduced algorithm, the lower-ranked task/particle carries out the computation. Process 0 computes f_{01} , f_{03} , and f_{23} , while process 1 computes f_{12} . As before, the newly computed forces are added into the appropriate locations in `loc_forces` and subtracted from the appropriate locations in `tmp_forces`.

To complete the algorithm, we need to exchange the `tmp` arrays one final time.¹ Once each process has received the updated `tmp_forces`, it can carry out a simple vector sum

```
loc_forces += tmp_forces
```

to complete the algorithm.

¹ Actually, we only need to exchange `tmp_forces` for the final communication.

```

1  source = (my_rank + 1) % comm_sz;
2  dest = (my_rank - 1 + comm_sz) % comm_sz;
3  Copy loc_pos into tmp_pos;
4  loc_forces = tmp_forces = 0;
5
6  Compute forces due to interactions among local particles;
7  for (phase = 1; phase < comm_sz; phase++) {
8      Send current tmp_pos and tmp_forces to dest;
9      Receive new tmp_pos and tmp_forces from source;
10     /* Owner of the positions and forces we're receiving */
11     owner = (my_rank + phase) % comm_sz;
12     Compute forces due to interactions among my particles
13         and owner's particles;
14 }
15 Send current tmp_pos and tmp_forces to dest;
16 Receive new tmp_pos and tmp_forces from source;

```

Program 7.3: Pseudocode for the MPI implementation of the reduced n -body solver.

Thus, we can implement the computation of the forces in the reduced algorithm using a ring pass with the pseudocode shown in Program 7.3.

Recall that using `MPI_Send` and `MPI_Recv` for the send-receive pairs in Lines 8–9 and 15–16 is *unsafe* in MPI parlance, since they can hang if the system doesn't provide sufficient buffering. In this setting, recall that MPI provides `MPI_Sendrecv` and `MPI_Sendrecv_replace`. Since we're using the same memory for both the outgoing and the incoming data, we can use `MPI_Sendrecv_replace`.

Also recall that the time it takes to start up a message is substantial. We can probably reduce the cost of the communication by using a single array to store both `tmp_pos` and `tmp_forces`. For example, we could allocate storage for an array `tmp_data` that can store $2 \times \text{loc_n}$ objects with type `vect_t` and use the first `loc_n` for `tmp_pos` and the last `loc_n` for `tmp_forces`. We can continue to use `tmp_pos` and `tmp_forces` by making these pointers to `tmp_data[0]` and `tmp_data[loc_n]`, respectively.

The principal difficulty in implementing the actual computation of the forces in Lines 12–13 lies in determining whether the current process should compute the force resulting from the interaction of a particle q assigned to it and a particle r , whose position it has received. If we recall the reduced algorithm (Program 7.1), we see that task/particle q is responsible for computing $\mathbf{f}_{qr}$ if and only if $q < r$. However, the arrays `loc_pos` and `tmp_pos` (or a larger array containing `tmp_pos` and `tmp_forces`) use *local* subscripts, not global subscripts. That is, when we access an element of (say) `loc_pos`, the subscript we use will lie in the range $0, 1, \dots, \text{loc_n} - 1$, not $0, 1, \dots, n - 1$; so, if we try to implement the force interaction with the following pseudocode, we'll run into (at least) a couple of problems:

```

for (loc_part1 = 0; loc_part1 < loc_n-1; loc_part1++) {
    for (loc_part2 = loc_part1+1;

```

```

        loc_part2 < loc_n;
        loc_part2++) {
    Compute_force(loc_pos[loc_part1], masses[loc_part1],
                  tmp_pos[loc_part2], masses[loc_part2],
                  loc_forces[loc_part1], tmp_forces[loc_part2]);
}
}

```

The first, and most obvious, is that `masses` is a global array and we're using local subscripts to access its elements. The second is that the relative sizes of `loc_part1` and `loc_part2` don't tell us whether we should compute the force due to their interaction. We need to use global subscripts to determine this. For example, if we have four particles and two processes, and the preceding code is being run by process 0, then when `loc_part1 = 0`, the inner loop will skip `loc_part2 = 0` and start with `loc_part2 = 1`; however, if we're using a cyclic distribution, `loc_part1 = 0` corresponds to global particle 0 and `loc_part2 = 0` corresponds to global particle 1, and we *should* compute the force resulting from interaction between these two particles.

Clearly, the problem is that we shouldn't be using local particle indexes, but rather we should be using *global* particle indexes. Thus, using a cyclic distribution of the particles, we could modify our code so that the loops also iterate through global particle indexes:

```

for (loc_part1 = 0, glb_part1 = my_rank;
    loc_part1 < loc_n-1;
    loc_part1++, glb_part1 += comm_sz) {
    for (glb_part2
        = First_index(glb_part1, my_rank, owner, comm_sz),
        loc_part2 = Global_to_local(glb_part2, owner, loc_n);
        loc_part2 < loc_n;
        loc_part2++, glb_part2 += comm_sz) {
        Compute_force(loc_pos[loc_part1], masses[glb_part1],
                      tmp_pos[loc_part2], masses[glb_part2],
                      loc_forces[loc_part1], tmp_forces[loc_part2]);
    }
}

```

The function `First_index` should determine a global index `glb_part2` with the following properties:

1. The particle `glb_part2` is assigned to the process with rank `owner`.
2. `glb_part1 < glb_part2 < glb_part1 + comm_sz`.

The function `Global_to_local` should convert a global particle index into a local particle index, and the function `Compute_force` should compute the force resulting from the interaction of two particles. We already know how to implement `Compute_force`. See Exercises 7.16 and 7.17 for the other two functions.

Table 7.5 Performance of the MPI n -body solvers (in seconds).

Processes	Basic	Reduced
1	17.30	8.68
2	8.65	4.45
4	4.35	2.30
8	2.20	1.26
16	1.13	0.78

Table 7.6 Run-times for OpenMP and MPI n -body solvers (in seconds).

Processes/ Threads	OpenMP		MPI	
	Basic	Reduced	Basic	Reduced
1	15.13	8.77	17.30	8.68
2	7.62	4.42	8.65	4.45
4	3.85	2.26	4.35	2.30

7.1.11 Performance of the MPI solvers

Table 7.5 shows the run-times of the two n -body solvers when they're run with 800 particles for 1000 timesteps on an Infiniband-connected cluster. All the timings were taken with one process per cluster node.

The run-times of the serial solvers differed from the single-process MPI solvers by less than 1%, so we haven't included them.

Clearly, the performance of the reduced solver is much superior to the performance of the basic solver, although the basic solver achieves higher efficiencies. For example, the efficiency of the basic solver on 16 nodes is about 0.95, while the efficiency of the reduced solver on 16 nodes is only about 0.70.

A point to stress here is that the reduced MPI solver makes much more efficient use of memory than the basic MPI solver; the basic solver must provide storage for all n positions on each process, while the reduced solver only needs extra storage for $n/\text{comm_sz}$ positions and $n/\text{comm_sz}$ forces. Thus, the extra storage needed on each process for the basic solver is nearly $\text{comm_sz}/2$ times greater than the storage needed for the reduced solver. When n and comm_sz are very large, this factor can easily make the difference between being able to run a simulation only using the process' main memory and having to use secondary storage.

The nodes of the cluster on which we took the timings have four cores, so we can compare the performance of the OpenMP implementations with the performance of the MPI implementations (see Table 7.6). We see that the basic OpenMP solver is a good deal faster than the basic MPI solver. This isn't surprising, since `MPI_Allgather` is such an expensive operation. Perhaps surprisingly, though, the reduced MPI solver is quite competitive with the reduced OpenMP solver.

Let's take a brief look at the amount of memory required by the MPI and OpenMP reduced solvers. Say that there are n particles and p threads or processes. Then each solver will allocate the same amount of storage for the local velocities and the local positions. The MPI solver allocates n **doubles** per process for the masses. It also allocates $4n/p$ **doubles** for the `tmp_pos` and `tmp_forces` arrays, so in addition to the local velocities and positions, the MPI solver stores

$$n + 4n/p$$

doubles per process. The OpenMP solver allocates a total of $2pn + 2n$ **doubles** for the forces and n **doubles** for the masses, so in addition to the local velocities and positions, the OpenMP solver stores

$$3n/p + 2n$$

doubles per thread. Thus, the difference in the local storage required for the OpenMP version and the MPI version is

$$n - n/p$$

doubles. In other words, if n is large relative to p , the local storage required for the MPI version is substantially less than the local storage required for the OpenMP version. So, for a fixed number of processes or threads, we should be able to run much larger simulations with the MPI version than the OpenMP version. Of course, because of hardware considerations, we're likely to be able to use many more MPI processes than OpenMP threads, so the size of the largest possible MPI simulations should be *much* greater than the size of the largest possible OpenMP simulations. The MPI version of the reduced solver is much more scalable than any of the other versions, and the "ring pass" algorithm provides a genuine breakthrough in the design of n -body solvers.

7.1.12 Parallelizing the basic solver using CUDA

When we apply Foster's method to parallelizing the n -body solvers for GPUs, the first three steps are the same as they were for the general parallelization in Subsection 7.1.3. So our composite or agglomerated tasks correspond to the individual particles in the simulation. As before, this will greatly reduce the number of inter-task communications.

Now, however, the mapping stage is somewhat different. In the MIMD implementations of the n -body solvers, we expected the number of tasks or particles to be much greater than the number of available cores, and for a very large problem this will probably be the case for a GPU-solver as well. However, recall that for GPUs, we usually *want* many more threads than SIMD cores (or datapaths) so that we can hide the latency of slow operations, such as loads and stores. So when we implement the mapping phase for a CUDA solver, it may make sense to have just one task or particle per thread. Let's look at implementing this approach—one particle per thread—and

we'll take a look at multiple particles per thread in the Programming Assignments. (See Programming Assignment 7.5.)

With this assumption each thread will compute something like this:

```
for each timestep  $t$  {
    Compute total force on my particle;
    Compute position and velocity of my particle;
}
```

However, there are a couple of race conditions in this pseudocode. Suppose threads A and B are executing the body of the loop for the same timestep. Also suppose that thread A updates the position of its particle *before* thread B starts the computation of the force on its particle. Then thread B will use the *wrong* value of the position of the particle assigned to thread A in the computation of the force on the particle assigned to B . So in any timestep no thread should proceed to the update of its particle's position and velocity until *after* all the threads have completed the updates to their particle's forces.

Now suppose that thread A has completed timestep t , and it has started computing the force on its particle in timestep $t + 1$. Also suppose that thread B is still executing timestep t , and it hasn't yet updated its particle's position. Then thread A will use the wrong value for thread B 's position in the computation of the force on its particle. So no thread should be able start timestep $t + 1$ until all the threads have completed timestep t .

Both of these race conditions can be prevented by the use of barriers. If we have a barrier across all the threads after the computation of the forces, then the first race condition can't occur, since thread A won't be able to proceed to the update of its particle's position and velocity until all the threads—including B —have completed the computation of the total force on their particles. Similarly, if we have a barrier after the updates to the positions and velocities, then no thread can proceed to timestep $t + 1$ until all the threads have completed their updates to the positions and velocities.

In Chapter 6 we saw that we could implement a barrier across *all* the threads by returning from one kernel and starting another. For example, suppose we wanted to place a barrier between calls to `Function_x` and `Function_y` in the following code:

```
/* Prototypes */
__device__ void Function_x (...);
__device__ void Function_y (...);
__global__ void My_kernel (...);
...
/* Host code */
...
My_kernel <<<blk_ct, th_per_blk>>> (...);

/* Device code */
__global__ void My_kernel (...) {
    Function_x (...);
    /* Want barrier here */
```

```

    Function_y (...);

    return;
}

```

Then we can rewrite our code so that `Function_x` and `Function_y` are kernels:

```

/* Prototypes */
__global__ void Function_x (...);
__global__ void Function_y (...);
...

/* Host code */
...
Function_x <<<blk_ct, th_per_blk>>> (...);
Function_y <<<blk_ct, th_per_blk>>> (...);
...

/* Device code */
__global__ void Function_x (...) {
    ...
}

__global__ void Function_y (...) {
    ...
}

```

In the second program the default behavior is for no thread to start executing `Function_y` until all the threads executing `Function_x` have returned.

Of course, in the general setting, there may be other code in `My_kernel`. In our case, the kernel with the race conditions might look something like this:

```

__global__ void Nbody_sim(vect_t forces[],
    struct particle_s curr[],
    double delta_t, int n, int n_steps) {
    int step;
    int my_particle = blkIdx.x*blkDim.x + threadIdx.x;

    for (step = 1; step <= n_steps; step++) {
        Compute_force(my_particle, forces, curr, n);
        Update_pos_vel(my_particle, forces, curr, n, delta_t);
    }
}

```

The only parts of the kernel that require the use of multiple threads are the calls to `Compute_force` and `Update_pos_vel`. So we can put the rest of the kernel in a *host* function²:

² In CUDA the `__host__` designation is the default. So we could omit it for this function.


```

__host__ void Nbody_sim(vect_t forces[],
    struct particle_s curr[],
    double delta_t, int n, int n_steps,
    int blk_ct, int th_per_blk) {
    int step;

    for (step = 1; step <= n_steps; step++) {
        Compute_force <<<blk_ct, th_per_blk>>> (
            forces, curr, n);
        Update_pos_vel <<<blk_ct, th_per_blk>>> (
            forces, curr, n, delta_t);
    }
    cudaDeviceSynchronize();
}

```

Now, instead of being device functions, `Compute_force` and `Update_pos_vel` are kernels, and instead of computing the particle indexes in `Nbody_sim`, we compute them in the new kernels.

In this code there is an implicit barrier before each call to `Update_pos_vel`. The call won't be started until all the threads executing `Compute_force` have returned. Also each call to `Compute_force` (except the first) will wait for the call to `Update_pos_vel` in the preceding iteration to complete.

The call to `cudaDeviceSynchronize` isn't needed, except as protection against accidentally executing host code that depends on the completion of the final call to `Update_pos_vel`. Recall that by default host code does *not* wait for the completion of a kernel.

7.1.13 A note on cooperative groups in CUDA

With the introduction of CUDA 9 and the Nvidia Pascal processor, it became possible to synchronize multiple thread blocks without starting a new kernel. The idea is to use a construct that is similar to an MPI communicator. In CUDA the new construct is called a **cooperative group**. A cooperative group consists of a collection of threads that can be synchronized with a function call that's made in a kernel or a device function.

To use cooperative groups, you must be running CUDA SDK 9 or later, and to use cooperative groups consisting of threads from more than one thread block, you must be using a Pascal processor or later.

Programming assignment 7.6 outlines how we might modify the basic CUDA solver so that it uses cooperative groups.

7.1.14 Performance of the basic CUDA n -body solver

Let's take a look at the performance of our basic CUDA n -body solver. We ran the CUDA program on an Nvidia-Pascal system (CUDA capability 6.1) with 20 SMs, 128 SPs per SM, and a 1.73 GHz clock. Table 7.7 shows the run-times. Every run was

Table 7.7 Run-times (in seconds) of basic CUDA n -body solver.

Blocks	Particles	Serial	CUDA
1	1024	7.01e-2	1.93e-3
2	2048	2.82e-1	2.89e-3
32	32,768	5.07e+1	7.34e-2
64	65,536	2.02e+2	2.94e-1
256	262,144	3.23e+3	2.24e+0
1024	1,048,576	5.20e+4	4.81e+1

with 1024 threads per block using `float` as the datatype. The serial times were taken on the host machine, which had an Intel Xeon Silver processor running at 2.1 GHz. Clearly, the CUDA implementation is *much* faster than the serial implementation. Indeed, the CUDA implementation is at least an order-of-magnitude faster, and it can be more than three orders of magnitude faster. Also note that the CUDA system is *much* more scalable than the serial implementation. As we increase the problem size, the run-times of the CUDA implementation usually increase much more slowly than the run-times of the serial implementation.

7.1.15 Improving the performance of the CUDA n -body solver

While the basic CUDA n -body solver is both faster and more scalable than the serial solver, its run-time *does* increase dramatically as the number of particles increases. For example, the time required for simulating 1,048,576 particles is nearly 25,000 times greater than the time required for 1024 particles. So we would like to further improve its performance.

If you’ve read the section on the OpenMP or Pthreads implementation, you’ll recall that for these implementations, we used temporary storage for the calculations of the forces involving particles assigned to a particular thread. This allowed us to improve the overall performance by halving the number of calculations carried out by each thread. However, the implementations allocated storage for

$$2 \times \text{thread_count} \times n$$

floating point values, and with n threads, this would require storage for $2n^2$ values. So if we were simulating a million particles, this would require storage for 2 *trillion* floating point values. If we use `floats`, this is 8 trillion bytes, and if we use `doubles`, it is 16 trillion bytes. We could probably reduce the amount of temporary storage by observing that we don’t need to store n values for each particle (see Tables 7.1 and 7.2), but this observation reduces the temporary storage by less than a factor of 2. (See Exercise 7.9.) So we would still need storage for more than 1 trillion bytes. At the time of this writing (January 2020), our most powerful Nvidia processor (Turing) has about 25 gigabytes of global storage. So we’d need storage that’s about 40 times larger than what’s available to us now.

Another possibility is to use shared memory.

7.1.16 Using shared memory in the n -body solver

Recall that in CUDA *global* memory is accessible to all the SMs and hence all the threads. However, the time required to load data from or store data to global memory is quite large compared to operations on data in registers (e.g., arithmetic operations). To compute the force on particle q , the basic solver must access the positions and masses of *all* of the particles. In the worst case, this will require that we access global memory for all n particles. Nvidia processors do have an L2 cache that is shared among all the SMs and a per-SM L1 cache. So some of these accesses will not require loads from global memory, but we can do better.

Recall that each SM has a block of on-chip memory called *shared memory*. The storage in the shared memory is shared among the threads in a thread block that's running on that SM. Since shared memory is on the same chip as an SM, it is relatively small, but since it is on-chip, it is *much* faster than global memory. So if we can arrange that many of our loads are from shared memory instead of global memory, we may be able to provide significant performance improvements.

Since shared memory is *much* smaller than global memory, we can't hope to use shared memory to store a copy of *all* the data we need from global memory. An alternative is to partition the data we need from global memory into logical "tiles" that are small enough to fit into shared memory. Then each thread block can iterate through the tiles. After the first tile is loaded into shared memory, a thread block carries out all of its computations that make use of the data in the first tile. Then the second tile is loaded into shared memory, and the thread block carries out all of its computations that make use of the data in the second tile, etc. This gives us the following pseudocode:

```
1   for each tile {  
2       Load tile into shared memory;  
3       Barrier across the thread block;  
4       Each thread carries out its computations  
5           making use of the data in this tile;  
6       Barrier across the thread block;  
7   }
```

Effectively, the tiles are being used as cache lines, except that accessing them is managed by our program instead of the hardware. The first barrier ensures that we don't start the calculations in Lines 4–5 until all of the data from the tile has been loaded. The second barrier ensures that we don't start overwriting the data in the current tile until all the threads have finished using it.

To apply this pseudocode to the n -body problem, we load the masses and positions of a collection of m particles in line 2, and then in Lines 4–5 a thread block will compute the forces between *its* assigned particles and the m particles the block has just loaded. So we need to determine m .

If each thread loads the position and mass of a single particle, we can execute the loads without thread divergence. Furthermore, CUDA currently limits the number of threads in a thread block to 1024, and if we load the positions and masses of 1024

particles, each thread will load three `floats` or three `doubles`, and we'll need shared memory storage for at most

$$3 \times 8 \times 1024 = 24 \text{ Kbytes.}$$

Since current-generation Nvidia processors have at least 48 Kbytes of shared memory, we can easily load the positions and masses of $m = \text{th_per_blk}$ particles from global memory into shared memory. Finally, it's easy and natural to choose the particles in a tile to have contiguous indexes. So our tiles will be

- Tile 0: positions and masses of particles 0 through $(\text{th_per_blk} - 1)$
- Tile 1: positions and masses of particles th_per_blk through $(2 \times \text{th_per_blk} - 1)$
- ...
- Tile k : positions and masses of particles $(k \times \text{th_per_blk})$ through $((k + 1) \times \text{th_per_blk} - 1)$
- ...
- Tile $(\text{blk_ct} - 1)$: positions and masses of particles $((\text{blk_ct} - 1) \times \text{th_per_blk})$ through $(\text{blk_ct} \times \text{th_per_blk} - 1)$

In other words, we're using a *block* partition of the positions and masses to form the tiles.

This gives us the following pseudocode:

```

1      // Thread q computes total force on particle q
2      Load position and mass of particle q
3      F_q = 0; /* Total force on particle q */
4      for each tile t = 0, 1, ..., blk_ct-1 {
5          // my_loc_rk = rank of thread in thread block
6          Load position and mass of particle
7              t*tile_sz + my_loc_rk into shared memory;
8          __syncthreads();
9
10         // All of the positions and masses for the particles in
11         //     tile t have been loaded into shared memory
12         for each particle k in tile t (except particle q) {
13             Load position and mass of k from shared memory;
14             Compute force f_qk of particle k on particle q;
15             F_q += f_qk;
16         }
17         // Don't start reading data from next tile until all
18         //     threads in this block are done with the current tile
19         __syncthreads();
20     }
21     Store F_q;
```

The code outside the `for` each tile loop is the same as the code outside the `for` each particle loop in the original basic solver: before the loop we load a position and a mass, and after the loop we store a force. The outer loop iterates through

the tiles, and, as we noted, each tile consists of `th_per_blk` particles. So each thread loads the position and mass of a single particle.

Before proceeding to the calculation of the forces, we call `__syncthreads()`. Recall that this is a barrier among the threads in a single thread block. So in any thread block, no thread will proceed to the computation of forces due to the particles in tile `t` until all of the data on these particles is in shared memory.

Once the data on the particles in tile `t` are loaded, each thread uses the inner **for** loop to compute the forces on its particle that are due to the particles in tile `t`. Note that if there are enough registers to store local variables, then all of these calculations will use data in registers or data in shared memory.

After each thread has computed the forces on its particle due to the particles in tile `t`, we have another call to `__syncthreads()`. This prevents any thread from continuing to the next iteration of the outer loop until all the threads in the thread block have completed the current iteration. This will ensure that the data on the particles in tile `t` isn't overwritten by data on particles in tile `t+1` until all the threads in the block are done with the data on the particles in tile `t`.

Now let's look at how many words are loaded and stored by the original basic CUDA solver and the CUDA shared memory solver in the computation of the forces in a single timestep. Both solvers start by having each thread load the position and mass of a single particle, and both solvers complete the computation of the forces by storing the total force on each particle. So if there are n particles, both solvers will start by loading $3n$ words—two for each position, and one for each mass—and they'll store $2n$ words: two for each force.

Since these are the same for both solvers, we can just compare the number of words loaded and stored in the main loop of each of the solvers.

In the basic solver, each thread will iterate through *all* of the particles, except the thread's own particle. So each thread will load $3(n - 1)$ words, and since we're assuming there are n threads, this will result in a total of $3n(n - 1)$ words.

In the shared memory solver, each tile consists of `th_per_blk` particles, and there are `blk_ct` tiles. So each thread in each block will load 3 words in Lines 6–7. Thus each execution of these lines by each of the threads will result in loading $3n$ words. Since there are `blk_ct` tiles, we get a total of

$$3n \times \text{blk_ct}$$

words loaded in the outer **for** loop. The loads in the inner **for** loop are from shared memory. So their cost is *much* less than the loads from global memory, and we have that the ratio of words loaded from global memory in the basic solver to the shared memory solver is

$$\frac{3n(n - 1)}{3n \times \text{blk_ct}} \approx \frac{n}{\text{blk_ct}} = \text{th_per_blk}.$$

So, for example, if we have 512 threads per block, then the original, basic solver will load approximately 512 times as many words from global memory as the shared memory solver.

Table 7.8 Run-times (in seconds) of CUDA basic and CUDA shared memory n -body solvers.

Blocks	Particles	Basic	Shared Mem	Speedup
1	1024	1.93e-3	1.59e-3	1.21
2	2048	2.89e-3	2.17e-3	1.33
32	32,768	7.34e-2	4.17e-2	1.76
64	65,536	2.94e-1	1.98e-1	1.48
256	262,144	2.24e+0	1.96e+0	1.14
1024	1,048,576	4.81e+1	2.91e+1	1.65

The actual performance of the basic and shared-memory solvers is shown in Table 7.8. We ran the shared-memory solver on the same system that we ran the basic solver on: an Nvidia Pascal-based system (CUDA capability 6.1) with 20 SMs, 128 SPs per SM, and a 1.73 GHz clock. Also, as with the basic solver, we always used 1024 threads per block. The shared memory implementation is always a good deal faster than the basic implementation, and on average, the speedup is 1.43.

7.2 Sample sort

7.2.1 Sample sort and bucket sort

Sample sort is a generalization of an algorithm known as *bucket sort*. In bucket sort, we're given two values $c < d$, and the keys are assumed to have a uniform distribution over the interval $[c, d)$. This means that if we take two subintervals $[p, q)$ and $[r, s)$ of the same length, then the number of keys in $[p, q)$ is approximately equal to the number of keys in $[r, s)$. So suppose we have n keys (often **floats** or **ints**), and we divide $[c, d)$ into b “buckets” or “bins” of equal length. Finally suppose that $h = (d - c)/b$. Then there should be approximately n/b keys in each of the buckets:

$$[c, c + h), [c + h, c + 2h), \dots, [c + (b - 1)h, b).$$

In bucket sort we first assign each key to its bucket, and then sort the keys in each bucket.

For example, suppose $c = 1$, $d = 10$, the keys are

$$9, 8, 2, 1, 4, 6, 3, 7, 2,$$

and we're supposed to use 3 buckets. Then $h = (10 - 1)/3 = 3$, and the buckets are

$$[1, 4), [4, 7), [7, 10).$$

So the contents of the three buckets should be

$$\{2, 1, 3, 2\}, \{4, 6\}, \text{ and } \{9, 8, 7\},$$

respectively.

In sample sort, we don't know the distribution of the keys. So we take a "sample" and use this sample to estimate the distribution. The algorithm then proceeds in the same way as bucket sort. For example, suppose we have b buckets and we choose our sample by picking s keys from the n keys. Here $b \leq s \leq n$. Suppose that after sorting the sample, its keys are

$$a_0 \leq a_1 \leq \dots \leq a_{s-1},$$

and we choose the keys in slots that are separated by s/b sample elements as "splitters." If the splitters are

$$e_1, e_2, \dots, e_{b-1}$$

then we can use

$$[\min, e_1), [e_1, e_2), \dots, [e_{b-1}, \max)$$

as our buckets.³ Here $\min$ is any value less than or equal to the smallest element in the list, and $\max$ is a value greater than the largest element in the list.

For example, suppose that after being sorted our sample consists of the following 12 keys:

$$2, 10, 23, 25, 29, 38, 40, 49, 53, 60, 62, 67.$$

Also suppose we want $b = 4$ buckets. Then the first three keys in the sample should go in the first bucket, the next three into the second bucket, etc. So we can choose as the "splitters"

$$e_1 = (23 + 25)/2 = 24, \quad e_2 = (38 + 40)/2 = 39, \quad e_3 = (53 + 60)/2 = 57.$$

Note that we're using the convention that if the splitter is not a whole number, we round up.

After determining the splitters, we assign each of the keys in the list to one of the buckets

$$[\min, 24), [24, 39), [39, 57), [57, \max).$$

Thus, we can use the pseudocode in Program 7.4 as the basis for an implementation of serial sample sort. Note that we're distinguishing between "sublists" of the input list and "buckets" in the sorted list. The sublists partition the input list into b sublists, each of which has n/b elements. The buckets partition the list into b subsets of elements with the property that all the keys in one bucket lie between consecutive splitters. So after each of the buckets has been sorted, we can concatenate them to get a sorted list.

³ If there are many duplicates in the list, we may want to allow the buckets to be closed intervals, and if duplicate elements are equal to a splitter, we can divide the duplicates between a pair of consecutive buckets. We won't consider this possibility in the programs we develop.

Clearly, to convert this pseudocode into a working program we must implement the sample generation function and the mapping function and any associated data structures. There are many possibilities. Let's take a look at some of them.

```

1  /* Input:
2   *   list[n]: the input list
3   *       n: number of elements in list (evenly
4   *           divisible by s and b)
5   *       s: number of elements in the sample (evenly
6   *           divisible by b)
7   *       b: number of buckets
8   * Output:
9   *   list[n]: the original input data in sorted order
10  */
11
12 /* Take the sample */
13 Gen_sample(list, n, s, b, sample);
14
15 Sort(sample, s);
16
17 FindSplitters(sample, s, splitters, b);
18
19 /* Assign each element of each sublist to
20  * its destination bucket in mapping, a data
21  * structure that can store all n elements.
22  */
23 Map(list, n, splitters, b, mapping);
24
25 /* Sort each bucket and copy it back to the original list */
26 Sort_buckets(mapping, n, b, list);

```

Program 7.4: Pseudocode for a serial sample sort.

7.2.2 Choosing the sample

An obvious method for generating the sample is to use a random number generator. For example,

```

// Seed the random number generator
srandom(...);
chosen = EMPTY_SET; // subscripts of chosen elements
for (i = 0; i < s; i++) {
    // Get a random number between 0 and n
    sub = random() % n;
    // Don't choose the same subscript twice
    while (Is_an_element(sub, chosen))

```



```

        sub = random() % n;

        // Add sub to chosen elements
        Add(chosen, sub);
        sample[i] = list[sub]
    }

```

The variable `chosen` is a set abstract data type that's used to store the subscripts of elements that have been added to the sample. The **while** loop is used to ensure that we don't choose the same subscript twice. For example, suppose `random() % n` is 5 when `i = 2` and when `i = 4`. Then without the **while** loop the sample will include element `list[5]` twice. Checking whether `sub` has already been chosen can be expensive, but if the sample size, `s`, is small relative to `n`, the size of the list, then it's unlikely that we'll choose the same subscript twice, and we'll only need to check `chosen` once for each element in the sample.

An alternative takes a “deterministic” sample.

```

for each sublist {
    Sort(sublist, n/b);

    Choose s/b equally spaced elements
    from the sorted sublist and add
    these to the sample;
}

```

For example, suppose that after being sorted a sublist consists of the elements {21, 27, 49, 62, 86, 92} and we're choosing two elements from each sublist for the sample. Then we could choose elements 49 and 92 from this sublist. Even if the sample size is large, the main cost of this method will be the calls to `Sort`. So this method will work best if there is a large number of relatively small sublists.

7.2.3 A simple implementation of the Map function

Perhaps the simplest implementation of the `Map` function allocates `b` arrays, each capable of storing at least `n/b` elements—for example, `2n/b` elements. This is the mapping data structure in the pseudocode in Program 7.4. These arrays will be the buckets. After allocating the buckets, we can simply iterate through the elements of `list` determining for each element which bucket it should be mapped to and copying the element to the appropriate bucket.

```

Allocate storage for b buckets;
for each element x of list {
    Search splitters for x's bucket;
    if (bucket is full)
        Allocate additional storage for
        bucket b;
    Append x to bucket b;
}

```

The catch is that, in general, a bucket may need to store more than the initially allocated number of elements, and we'll need to increase its size. We can do this using the C library function `realloc`:

```
void* realloc(void* ptr, size_t size);
```

For us, `ptr` will be one of the buckets, and `size` is the amount of storage, in bytes, that we want for the resized bucket. If there is insufficient space immediately following the memory that has already been allocated, `realloc` will try to find another block that is large enough and copy the contents of the old block into the new block. When this happens, the pointer to the new block is returned by the call. If this isn't necessary, `realloc` returns the old pointer. If there isn't enough memory on the heap for the new allocation, `realloc` returns `NULL`. Note that any memory referred to by `ptr` before the call to `realloc` *must* have been allocated on the heap.

A final note here: `realloc` can be an expensive call. So in general, we don't allocate storage for just one additional element, since there's a good chance that we'll need to allocate more memory for the same bucket. Of course, in this setting, we don't know how much more memory we will ultimately need. If you'll be using the sample sort on a particular type of data, it may be useful to run some tests on data that is similar to the data encountered in the application that will be using the sort. However, for a fully general sort, we won't have this option. So we might try allocating another n/b elements when a bucket is full.

7.2.4 An alternative implementation of Map

An alternative implementation of Map uses a single n -element array, `temp`, for the buckets, but it builds some additional data structures that provide information about the buckets. These data structures might store the following information:

1. How many elements will be moved from each sublist to each bucket, `from_to_cts`;
2. The subscript of the first element in each sublist that goes to each bucket `bkt_starts_in_slists`;
3. The subscript of the first element in each bucket that comes from each sublist `slist_starts_in_bkts`, and the total number of elements in each bucket;
4. The location of the first element of each bucket in `temp`, `bkt_starts`.

The first two data structures are $b \times b$ arrays with rows corresponding to sublists and columns corresponding to buckets. The third data structure is a $(b + 1) \times b$ array with rows $0, 1, \dots, b - 1$ corresponding to sublists, columns corresponding to buckets, and the last row storing the total number of elements in each bucket. The fourth data structure is a b -element array.

Note that we may not need all of these data structures. For example, in our serial implementation, we only use the first b rows of `slist_starts_in_bkts` to compute the last row. So we could simply sum each column of `from_to_cts` to get the final row and store it in a one-dimensional array.

We can build these data structures using the following pseudocode:

```

Sort each sublist;
for each sublist
    for each bucket
        Count elements in sublist
            going to bucket and store
            in from_to_cts[sublist,bucket];
for each sublist
    Build bkt_starts_in_slists
        by forming exclusive prefix sums
        of row sublist of from_to_cts;
for each bucket
    Build sublist_starts_in_bkts
        by forming exclusive prefix sums
        of column bucket of from_to_cts;
Build bkt_starts by forming
    exclusive prefix sums of last row
    of sublist_starts_in_bkts;

```

In spite of the apparent complexity of this code, it only uses **for** loops and three basic algorithms:

1. A sort for the sublists,
2. Counting elements of a sublist that fall in a range determined by the splitters, and
3. Exclusive prefix sums of a row or column of a two-dimensional array. (See paragraph ‘Prefix sums’ on p. 404.)

For the sort, we should probably choose a sorting algorithm that has relatively little overhead, so that it will be quite fast on short lists. So let’s look at some details of the second and third algorithms.

To illustrate the second algorithm, we need to specify the details of the organization of the array of splitters. We’ll assume that there are $b + 1$ elements in the array `splitters` and they’re stored as shown in Table 7.9. The constants `MINUS_INF` and `INF` have been chosen so that they are, respectively, less than or equal to and greater than any possible element of the input list.

Since the sublists have been sorted, we can implement the second item by iterating sequentially through a sublist:

```

int curr_row[b]; /* row of from_to_cts */
dest_bkt = 0; /* subscript in splitters */

Assign(curr_row, 0); /* Set elements to 0 */
for (j = 0; j < n/b; j++) {
    while (sublist[j] >= splitters[dest_bkt+1])
        dest_bkt++;
    curr_row[dest_bkt]++;
}

```

Table 7.9 The splitters in sample sort.

Bucket	Elements
0	$\text{MINUS_INFTY} = \text{splitters}[0] \leq \text{key} < \text{splitters}[1]$
1	$\text{splitters}[1] \leq \text{key} < \text{splitters}[2]$
$\vdots$	$\vdots$
i	$\text{splitters}[i] \leq \text{key} < \text{splitters}[i+1]$
$\vdots$	$\vdots$
$b-1$	$\text{splitters}[b-1] \leq \text{key} < \text{splitters}[b] = \text{INFTY}$

So we iterate through the elements of the current sublist in the **for** statement. For each of these elements, we iterate through the remaining splitters until we find a splitter that is strictly greater than the current element.

Suppose, for example, that $n = 24$, $b = 4$, and the splitters are $\{-\infty, 45, 75, 91, \infty\}$. Now suppose one of the sublists contains the elements $\{21, 27, 49, 62, 86, 92\}$. Then both 21 and 27 are less than 45. So the first two elements will go to bucket 0. Similarly, 49 and 62 are both less than 75 (and greater than 45). So they should go to bucket 1. However $86 > 75$, but less than 91; so it should go to bucket 2. Finally, $92 > 91$, but less than ∞ ; and it should go to bucket 3. This gives us two elements going to bucket 0, two elements going to bucket 1, one element going to bucket 2, and one element going to bucket 3.

Note that since the splitters are sorted, the linear search in the while loop could be replaced with binary search. Alternatively, since the sublists are sorted, we could use a binary search to find the location of each splitter in the sublist. (See Exercise 7.20.)

Prefix sums

A **prefix sum** is an algorithm that is applied to the elements of an n -element array of numbers, and its output is an n -element array that's obtained by adding up the elements "preceding" the current element. For example, suppose our array contains the elements

5, 1, 6, 2, 4

Then an *inclusive* prefix sum of this array is

5, 6, 12, 14, 18

So

```
incl_pref_sum[0] = array[0];
```

and for $i > 0$

```
incl_pref_sum[i] = array[i] + incl_pref_sum[i-1];
```

An *exclusive* prefix sum of our array is

0, 5, 6, 12, 14

So in an exclusive prefix sum, element 0 is 0, and the other elements are obtained by adding the *preceding* element of the array to the most recently computed element of the prefix sum array:

```
excl_pref_sum[0] = 0;
```

and for $i > 0$

```
excl_pref_sum[i] = array[i-1] + excl_pref_sum[i-1];
```

Observe that since the sublists have been sorted, the exclusive prefix sums of the rows of `from_to_cts` tell us which element of a sublist should be the first element from the sublist in each bucket. In our preceding example, we have $b = 4$ buckets and for the sublist {21, 27, 49, 62, 86, 92} the row corresponding to it in `from_to_cts` is

2, 2, 1, 1.

The exclusive prefix sums of this array are

0, 2, 4, 5

So the subscripts of the first element in the sublist going to each of the four buckets are shown in the following table:

Bucket	Subscript of First Element Going to Bucket	First Element Going to Bucket
0	0	21
1	2	49
2	4	86
3	5	92

In fact, the elements of each of the remaining data structures can be computed using exclusive prefix sums. Here's a complete example with $n = 24$, $b = 4$, $s = 8$, and the following list:

```
Sublist 0: 15, 35, 77, 83, 86, 93
Sublist 1: 21, 27, 49, 62, 86, 92
Sublist 2: 26, 26, 40, 59, 63, 90
Sublist 3: 11, 29, 36, 67, 68, 72
```

As before, the splitters are $\{-\infty, 45, 75, 91, \infty\}$. Using this information, the various data structures are

from_to_cts				
sublist	bucket			
	0	1	2	3
0	2	0	3	1
1	2	2	1	1
2	3	2	1	0
3	3	3	0	0

bkt_starts_in_slists				
sublist	bucket			
	0	1	2	3
0	0	2	2	5
1	0	2	4	5
2	0	3	5	6
3	0	3	6	6

slist_starts_in_bkts				
sublist	bucket			
	0	1	2	3
0	0	0	0	0
1	2	0	3	1
2	4	2	4	2
3	7	4	5	2
bkt sizes	10	7	5	2

bkt_starts				
bucket	0	1	2	3
1st elt	0	10	17	22

Note that the last row of `slist_starts_in_bkts` is the number of elements in each bucket, and it is obtained by continuing the prefix sums down the columns of `from_to_cts`. Finally, note that `bkt_starts` is obtained by taking the exclusive prefix sums of the last row of `slist_starts_in_bkts`, and it tells us where each bucket should start in the array `temp`. So we don't need to do any reallocating in this version, and there's no wasted space.

Finishing up

Now we can assign elements of `list` to `temp`, the array that will store the buckets. Since `bkt_starts` tells us where the first element of each bucket is (e.g., the first element in bucket 1 should go to `temp[10]`), we can finish by iterating through the sublists, and in each sublist iterating through the buckets:

```

1   for each slist of list {
2       sublist = pointer to start of next block
3           of n/b elements in list;
```

```

4      for each bkt in temp {
5          bucket = pointer to start of bkt in temp;
6          dest = 0;
7
8          /* Now iterate through the elements of sublist */
9          first_src_sub = bkt_starts_in_slist[slist, bkt];
10         if (bkt < b)
11             last_src_sub = bkt_starts_in_slist[slist, bkt+1];
12         else
13             last_src_sub = n/b;
14         for (src = first_src_sub; src < last_src_sub; src++)
15             bucket[dest++] = sublist[src];
16     }
17 }
```

The assignments in Lines 2 and 4 are just defining pointers to the beginning of the current sublist and the beginning of the current bucket. The innermost loop iterates through the elements of the current sublist that go to the current bucket. In general, the limits of this iteration can be obtained from `bkt_starts_in_slist`. However, since there are only b columns in this array, the last element has to be treated as a special case—hence the else clause.

7.2.5 Parallelizing sample sort

Serial sample sort can be divided into a sequence of well-defined phases:

1. Take a sample from the input list.
2. Find the splitters from the sample.
3. Build any needed data structures from the list and the splitters.
4. Map keys in the list into buckets.
5. Sort the buckets.

There are other possible phases—e.g., sorts at various points in the algorithm—and, as we’ve already seen, there are a variety of possible implementations of each phase.

Serial sample sort can be naturally parallelized for MIMD systems by assigning sublists and buckets to processes/threads. For example, if each process/thread is assigned a sublist, then Steps 1 and 2 can have embarrassingly parallel implementations. First each process/thread chooses a “subsample” from its sublist, and then the splitters are chosen by each process/thread from its subsample.

Since CUDA thread blocks can operate independently of each other, in CUDA it is natural to assign sublists and buckets to thread blocks instead of individual threads.

We’ll go into detail for the various APIs below, but we’ll assume that each MIMD process/thread is assigned a sublist and a bucket. For the CUDA implementations, we’ll look at two different assignments. The first, basic, implementation will also assign a bucket and a sublist to each thread. We’ll use a more natural, but more complex, approach in the second implementation, where we’ll assign buckets and sublists to thread blocks.

Note also that in our serial implementations, we used a generic sort function several times. So for the MIMD parallel implementations, we'll assume that the sorts are either single-threaded or that we have a parallel sort function that performs well on a relatively small list. For the basic CUDA implementation, we'll use single-threaded sorts on the host and each thread. For the second implementation, we'll assume that we have a parallel sort function that performs well when implemented with a single thread block and one that works well with multiple thread blocks on relatively small lists.

Finally, for our “generic” parallelizations, we'll focus on the MIMD APIs, and we'll address the details of CUDA implementations in Section 7.2.9.

Basic MIMD implementation

The first implementation uses many of the ideas from our first serial implementation:

1. Use a random number generator to choose a sample from the input list.
2. Sort the sample.
3. Choose splitters from the sample.
4. Use the splitters to map keys to buckets.
5. Sort buckets.

As we noted earlier, Step 1 can be embarrassingly parallel if we assign a sublist to each MIMD process/thread. For Step 2 we can either have a single process/thread carry out the sort, or we can use a parallel sort that is reasonably efficient on short lists (e.g., parallel odd-even transposition sort).

If the sample is distributed among the processes/threads, then Step 3 can be implemented by having each process/thread choose a single splitter. So if we used a single process/thread for Step 2, we can first distribute the sample among the processes/threads. Finally, to determine which elements of the list go to which buckets, each process/thread will need access to all of the splitters. So in a shared memory system, the splitters should be shared among the threads, and in a distributed memory system, the splitters should be gathered to each process.

For Step 4, one task might be to determine which bucket an element of the list should be mapped to. A second might be to assign the element to its destination bucket. Clearly these two tasks can be aggregated, since the second can't proceed until the first is completed. The obvious problem here is that we need some sort of synchronization among tasks with the same destination bucket. For example, on a shared memory system, we might use a temporary array to record for each element its destination bucket. Then a process/thread can iterate through the list, and when it finds an element going to its assigned bucket, it can copy from the element its destination bucket. This will require an extra array to record destinations, but if the list isn't huge, this should be possible.

For distributed memory, this approach could be very expensive: we might gather both the original list and the list of destination buckets to each process. A less costly alternative would be to construct some of the data structures outlined in the second serial implementation and use these with an MPI collective communication called

`MPI_Alltoallv` to redistribute the elements of the list from each process to its correct destination. We'll go into detail in the MPI section (Section 7.2.8), but the idea is that each process has a block of elements going to every other process. For example, process q has some elements that should go to process 0, some that should go to process 1, etc., and this is the case for each process. So we would like for each process to scatter its original elements to the correct processes, *and* we would like for each process to gather its “final” elements from every process.

In either the shared- or the distributed-memory implementation, we can finish by having each process/thread sort its elements using a serial sort.

A second implementation

As you might have guessed our second implementation uses many of the ideas from the second serial implementation.

1. Sort the elements of each sublist.
2. Choose a random or a “deterministic” sample as discussed in our serial implementation.
3. Sort the sample.
4. Choose equally spaced splitters from the sample.
5. Use the input list and the splitters to build the data structures needed to implement the assignment of elements to buckets.
6. Assign elements of the list to the correct buckets.
7. Sort each bucket.

As with the first implementation, we'll assign a single sublist and a single bucket to each thread/process.

As we noted near the beginning of this section, for steps 1, 3, and 7 we assume each sort is either done by a single thread/process, or that we already have a good parallel sort for “small” lists.

Step 2 is embarrassingly parallel for both the random and deterministic samples. For the deterministic sample, a task is choosing an element of the sample, and since they're equally spaced, a single thread/process can choose a subset of the sample elements from a single sublist.

Step 4 is the same as choosing the splitters in the first parallel implementation.

For Step 5, there are two basic parts. First we need to count the number of elements going from each sublist to each bucket. So as before, one task might be determining for an element of the list which bucket it should go to. Now a second task could be incrementing the appropriate element of the data structure. Clearly these tasks can be aggregated. Since the number of processes/threads is the same as the number of buckets, we can map the aggregate tasks for the elements of a sublist to a single process/thread.

The second part of Step 5 is implementing one or more prefix sums. Since the number of processes/threads is the same as the number of buckets, one process/thread can be responsible for each set of prefix sums. See below and, for example, [27].

In Step 6, a single task can be to assign an element of a sublist to its bucket. We aggregate the tasks so that a single process/thread is responsible for a sublist or a bucket. So a process/thread can “push” the keys in its sublist to their destination buckets. Or, alternatively, since a process/thread is responsible for a single bucket, it can “pull” all the keys from the original list that go to its bucket.

7.2.6 Implementing sample sort with OpenMP

Our OpenMP implementation of the sample sort will assume that the list to be sorted is stored in memory as a contiguous array. After the sort is complete, the values will have been moved to their appropriate locations within the array. The number of elements in the list, n , and the sample size, s , must be evenly divisible by the number of threads, `thread_count` (as specified by `OMP_NUM_THREADS`).

First implementation

For our initial implementation, the main thread will be responsible for setting up data structures and performing any serial steps before the sort begins. This will include sampling the list, sorting the sample, and calculating the splitters. We’ll use the first approach described in Section 7.2.2 to determine which values to sample from the list by selecting a set of unique random numbers that correspond to the list indexes of the samples. Once the sample has been taken, we will sort it with the C library `qsort` function, since it is reasonably efficient with small lists. With the sample sorted, we calculate `thread_count + 1` splitters and use them to define a range of values that will be assigned to each thread’s bucket. The buckets are assigned sequentially across the threads, with thread 0 managing the smallest values (`MINUS_INFINITY` to the first splitter) and the last thread managing the largest values (from the last splitter to `INFINITY`). Once these steps are complete, the remainder of the algorithm runs in an OpenMP parallel region.

Each thread’s bucket is represented as an array of elements that fall within its splitter range and a global *bucket count* to track bucket sizes. We can use a slightly modified version of the simple approach outlined in Section 7.2.3 to populate the buckets. Following is the pseudocode:

```

1      for each element x of list {
2          if (x in bucket range) {
3              if (bucket is full) {
4                  Resize bucket;
5              }
6
7              Append x to bucket;
8              Increment global bucket count;
9          }
10     }
```

Since resizing a bucket requires an expensive call to `realloc`, buckets are initially allocated with enough space for $n/\text{thread_count}$ elements. If a resize operation is

required, the bucket size is doubled. This helps reduce calls to `realloc` at the expense of somewhat higher overall memory consumption.

Once the buckets are populated, threads sort their individual buckets via the C library `qsort` function. Since the bucket sizes vary from thread to thread, a barrier ensures that the bucket counts have all been calculated before proceeding to the final step: copying each thread's bucket back to the appropriate positions in the original list. Since the list is shared among all the threads, this final operation can be carried out in parallel. Here's complete pseudocode for the multi-threaded portion of the algorithm:

```

1  # pragma omp parallel
2  {
3      bucket_counts[rank] = 0;
4      my_bucket_size = n / thread_count;
5      my_bucket = malloc(my_bucket_size * sizeof(int));
6
7      // Retrieve bucket range from the global splitter list
8      bucket_low, bucket_high = Calc_splitter_range(rank);
9
10     for each element x of list {
11         if (x >= bucket_low && x < bucket_high) {
12             if (bucket is full) {
13                 my_bucket_size = my_bucket_size * 2;
14                 my_bucket = realloc(
15                     my_bucket, my_bucket_size);
16             }
17
18             // Append x to the bucket, then
19             //      increment global bucket count
20             my_bucket[bucket_counts[rank]++] = x;
21         }
22     }
23
24     // 0 = subscript of first element
25     Sort(my_bucket, 0, my_bucket_size);
26
27     // Wait for all threads to sort
28     # pragma omp barrier
29
30     // Determine bucket positions using
31     //      global bucket counts
32     list_position = Calc_list_position(
33         rank, bucket_counts[rank])
34
35     // Copy sorted sublist from bucket
36     //      back to the global list
37     Copy(list + list_position, my_bucket, my_bucket_size)
38 }

```

Table 7.10 shows run-times (in seconds) of our initial implementation on a system with dual AMD EPYC 7281 16-Core Processors. The input list has $2^{24} = 16,777,216$ ints, and the sample has 65,536 ints. The run-time of the C library `qsort` is also provided for comparison. The program was compiled with compiler optimizations enabled (`gcc -O3`), 1000 trials were run for each test configuration, and the minimum time is reported.

Table 7.10 Run-times of the first OpenMP implementation of sample sort (times are in seconds).

Threads	Run-time
1	3.54
2	2.29
4	1.69
8	1.34
16	1.40
32	1.36
64	1.33
<code>qsort</code>	2.59

While the run-times reported in Table 7.10 demonstrate an improvement over the serial version of the algorithm, the implementation has some inefficiencies and clearly does not scale. One of the most significant issues with this approach is that every thread inspects the entire input list, resulting in a large amount of duplicated effort. As described previously in Section 7.2.4, we can avoid this overhead by modifying our `Map` implementation. Additionally, our sampling approach is limited to a single thread and may be inefficient with larger sample sizes due to repeated iterations through the list of “chosen” elements (see Section 7.2.2).

Second implementation

To improve the performance of our initial implementation, we take additional preparatory steps to populate the data structures described in Section 7.2.4. This ultimately decreases the amount of duplicated work performed by the threads and eliminates the need to reallocate space for buckets. While each thread in our first implementation “pulled” relevant values from the global list into its bucket based on its splitters, the second implementation “pushes” values from each bucket to their appropriate positions in the global list. Consequently, our second approach should be faster, and the difference in performance between the two approaches will increase as the list size increases.

Our first task is to divide the global list evenly among the threads, and then sort the values under their purview in place. We can determine the sublist indexes as follows:

```
my_sublist_size = n / thread_count;
my_first = my_rank * my_sublist_size;
my_last = my_first + my_sublist_size;
```

And then sort the elements from `my_first` through `my_last` in parallel.

To accelerate the sampling process, we use the “deterministic” sampling algorithm described in Section 7.2.2. We begin by calculating individual “subsample” ranges for each thread to populate:

```
subsample_size = s / thread_count;
subsample_idx = subsample_size * my_rank;
```

Each thread samples from the original list by deterministically choosing elements at a fixed interval. A barrier ensures all the threads have generated their portion of the sample, and then the first thread is responsible for sorting it and choosing the splitters. Note that this process is the only portion of the second implementation that is completed by a single thread.

```
// Use the deterministic sampling approach
Choose_sample(
    my_first, my_last, subsample_idx, subsample_size);

# pragma omp barrier

if (my_rank == 0) {
    Sort(sample, 0, s)
    splitters = Choose_splitters(sample, s, thread_count);
}

# pragma omp barrier
```

Once the sublists are sorted and the splitters have been chosen, each thread can determine the number of elements in its bucket that belong to each of the other threads (based on their particular splitters). These counts are stored in a global $n \times n$ *count matrix*, with each thread responsible for initializing one row of the matrix. Once all of the threads have completed this step, we can determine the number of elements that will be placed in each thread’s bucket by summing the corresponding column in the count matrix. Next, we allocate memory for the buckets and calculate the exclusive prefix sums for each row of the count matrix; this provides the indices for the start and end of each thread’s bucket within the global list.

Once the preparatory work is complete and our data structures have been populated, each thread will “pull” in values from the other threads by merging them into its bucket. After the process is complete, the buckets are copied to their appropriate locations within the global list to conclude the parallel sort. The full pseudocode for the parallel portion of this approach follows:

```
1    my_sublist_size = n / thread_count;
2    my_first = my_rank * my_sublist_size;
3    my_last = my_first + my_sublist_size;
4    subsample_size = s / thread_count;
5    subsample_idx = subsample_size * my_rank;
6
```

```

7      // Sort this thread's portion of the global list
8      Sort(list, my_first, my_last);
9
10     // Use the deterministic sampling approach
11     Choose_sample(my_first, my_last,
12                  subsample_idx, subsample_size);
13
14 #   pragma omp barrier
15
16     // This portion is handled by a single thread
17     if (my_rank == 0) {
18         Sort(sample, 0, s);
19         splitters = Choose_splitters(sample, s, thread_count);
20     }
21
22 #   pragma omp barrier
23
24     // Count the number of elements in this thread's sublist
25     // that will be pushed to each thread, stored as a row
26     // of global count_matrix
27     Find_counts(my_rank);
28 #   pragma omp barrier
29
30     // Find the bucket size for this thread
31     bucket_size = 0;
32     for each thread_rank {
33         idx = thread_rank * thread_count + my_rank;
34         bucket_size += count_matrix[idx];
35     }
36 #   pragma omp barrier
37
38     // Each thread calculates exclusive prefix sums for its row
39     Calc_prefix_sums();
40 #   pragma omp barrier
41
42     // Preparation complete. Begin merging:
43     for each thread_rank {
44         Merge(my_bucket, thread_rank);
45     }
46 #   pragma omp barrier
47
48     // Determine bucket positions using global bucket counts
49     list_low, list_high = Calc_list_positions(
50         rank, bucket_counts[rank]);
51
52     // Copy sorted sublist from bucket back to the global list
53     Copy(global_list, my_bucket, list_low, list_high);

```

Here, the `Merge` process is more or less the same as one would find in a standard merge sort: assuming both lists are sorted in ascending order, compare the “heads” of each list and move the smaller of the two heads into a temporary array. Once the end of either list is reached, the remaining values of the other list can be copied directly into the array.

Table 7.11 shows run-times (in seconds) of our second implementation on the same testbed with input list of $2^{24} = 16,777,216$ ints and sample of 65,536 ints. Again, the run-time of the C library `qsort` is also provided for comparison, and the program was compiled with compiler optimizations enabled (`gcc -O3`); 1000 trials were run for each test configuration, and the minimum time is reported.

Table 7.11 Run-times of the second OpenMP implementation of sample sort (times are in seconds).

Threads	Run-time
1	2.65
2	1.33
4	1.01
8	0.35
16	0.21
32	0.15
64	0.13
<code>qsort</code>	2.59

While the second approach requires a bit more upfront effort (both computationally and programmatically), it provides better run-times across the board, as each thread no longer iterates through the entire list of elements.

7.2.7 Implementing sample sort with Pthreads

As was the case with the two n -body solvers, the Pthreads implementation of sample sort is very similar to the OpenMP implementation. To avoid repeating the previous discussion, we will focus on the differences between the two approaches.

For our first iteration of the algorithm, we can replace OpenMP parallel regions with thread procedures. After the initial steps are completed by the main thread (gathering parameters, allocating data structures, and choosing the sample and splitters), we use `pthread_create` to start the parallel portion of the algorithm. The pseudocode follows:

```

1   bucket_counts[rank] = 0;
2   my_bucket_size = n / thread_count;
3   my_bucket = malloc(my_bucket_size * sizeof(int));
4
5   // Retrieve bucket range from the global splitter list
6   bucket_low, bucket_high = Calc_splitter_range(rank);

```

```

7
8   for each element x of list {
9       if (x >= bucket_low && x < bucket_high) {
10           if (bucket is full) {
11               my_bucket_size = my_bucket_size * 2;
12               my_bucket = realloc(my_bucket, my_bucket_size);
13           }
14
15           // Append x to bucket, increment global bucket count
16           my_bucket[bucket_counts[rank]++] = x;
17       }
18   }
19
20   Sort(my_bucket, 0, my_bucket_size);
21
22   // Wait for all threads to sort
23   pthread_barrier_wait(&bar);
24
25   // Determine bucket positions using global bucket counts
26   list_position = Calc_list_position(
27       rank, bucket_counts[rank])
28
29   // Copy sorted sublist from bucket back to the global list
30   Copy(list + list_position, my_bucket, my_bucket_size)

```

Note that the primary difference here is we are using a barrier from the Pthreads library. Usage is typical of Pthreads objects:

```

1   // Shared barrier variable
2   pthread_barrier_t bar;
3
4   ...
5
6   // Initialize barrier 'bar' with n threads
7   // (done once during the serial portion of the algorithm):
8   pthread_barrier_init(&bar, NULL, n);
9
10  ...
11
12  // Wait for all threads to reach the barrier:
13  pthread_barrier_wait(&bar);
14
15  ...
16
17  // Destroy the barrier in the main thread, when it's
18  // no longer needed:
19  pthread_barrier_destroy(&bar);
20  //

```


See Exercise 4.10 in Chapter 4.

However, since Pthread barriers are not available on all platforms, we can also build our own implementation with a condition variable:

```

1  void Barrier(void) {
2      pthread_mutex_lock(&bar_mut);
3      bar_count++;
4      if (bar_count == thread_count) {
5          bar_count = 0;
6          pthread_cond_broadcast(&bar_cond);
7      } else {
8          while (pthread_cond_wait(&bar_cond, &bar_mut) != 0);
9      }
10     pthread_mutex_unlock(&bar_mut);
11 }
```

Here `bar_mut` and `bar_cond` are shared variables. See Subsection 4.8.3 in Chapter 4 for further details.

Table 7.12 shows run-times (in seconds) of our initial implementation on a system with dual AMD EPYC 7281 16-Core Processors. The input list has $2^{24} = 16,777,216$ ints, and the sample has 65,536 ints. The run-time of the C library `qsort` is also provided for comparison. The program was compiled with compiler optimizations enabled (`gcc -O3`); 1000 trials were run for each test configuration, and the minimum time is reported.

Table 7.12 Run-times of the first Pthreads implementation of sample sort (times are in seconds).

Threads	Run-time
1	3.55
2	2.31
4	1.71
8	1.48
16	1.45
32	1.28
64	1.24
qsort	2.59

Just as with the OpenMP version, this implementation provides some improvement but is clearly not scalable. We applied the same optimizations to the second implementation (parallel deterministic sample, determining bucket sizes in advance, pulling values into the buckets and merging). Table 7.13 shows the run-times (in seconds) on the same testbed with input list of $2^{24} = 16,777,216$ ints and sample of 65,536 ints. Again, the run-time of the C library `qsort` is also provided for comparison and the program was compiled with compiler optimizations enabled (`gcc -O3`).

Table 7.13 Run-times of the second Pthreads implementation of sample sort (times are in seconds).

Threads	Run-time
1	2.65
2	1.36
4	1.08
8	0.35
16	0.20
32	0.14
64	0.12
qsort	2.59

We can observe that performance is very similar for both the OpenMP and Pthreads approaches. This is not surprising; apart from some minor syntactical differences, their implementations are also quite similar.

7.2.8 Implementing sample sort with MPI

For the MPI implementation, we assume that the initial list is distributed among the processes, but the final, sorted list is gathered to process 0. We also assume that both the total number of elements n and the sample size s are evenly divisible by the number of processes p . So each process will start with a sublist of n/p elements, and each process can choose a “subsample” of s/p elements.

First implementation

Each process can choose a sample of its sublist using the method outlined above for the first serial algorithm. Then these can be gathered onto process 0. Process 0 will sort the sample, choose the splitters, and broadcast the splitters to all the processes. Since there will be p buckets, there will be $p + 1$ splitters: the smallest less than or equal to any element in the list (`MINUS_INF`), and the largest greater than any element in the list (`INF`).

Then each process can determine to which bucket each of its elements belongs, and we can use the MPI function `MPI_Alltoallv` to distribute the contents of each sublist among the buckets. The syntax is

```
int MPI_Alltoallv(
    const void *send_data      /* in */,
    const int *send_counts     /* in */,
    const int *send_displs     /* in */,
    MPI_Datatype send_type     /* in */,
    void *recv_data           /* out */,
    const int *recv_counts     /* in */,
    const int *recv_displs     /* in */,
```

```

MPI_Datatype  recv_type    /* in */,
MPI_Comm      comm        /* in */);

```

In our setting, `send_data` is the calling process' sublist, and `recv_data` is the calling process' bucket. Furthermore, the function assumes that the elements going to any one process occupy contiguous locations in `send_data`, and the elements coming from any one process should occupy contiguous locations in `recv_data`.⁴ The communicator is `comm`, which for us will be `MPI_COMM_WORLD`. For us, both `send_type` and `recv_type` are `MPI_INT`. The arrays `send_counts` and `send_displs` specify where the elements of the sublist should go, and the arrays `recv_counts` and `recv_displs` specify where the elements of the bucket should come from.

As an example, suppose we have three processes, the splitters are $-\infty, 4, 10, \infty$ and the sublists and buckets are shown in the following table:

	Processes		
	0	1	2
sublists	3, 5, 6, 9	1, 2, 10, 11	4, 7, 8, 12
buckets	3, 1, 2	5, 6, 9, 4, 7, 8	10, 11, 12

Then we see that processes should send the following numbers of elements to each process:

From Process	To Process		
	0	1	2
0	1	3	0
1	2	0	2
2	0	3	1

So `send_counts` on each process should be

Process 0 : {1, 3, 0}

Process 1 : {2, 0, 2}

Process 2 : {0, 3, 1}

and `recv_counts` should be

Process 0 : {1, 2, 0}

Process 1 : {3, 0, 3}

Process 2 : {0, 2, 1}

So `recv_counts` can be obtained by taking the *columns* of the matrix whose rows are the `send_counts`.

⁴ These assumptions apply when the datatype arguments occupy contiguous locations in memory.

The array `send_counts` tells `MPI_Alltoallv` how many `ints` should be sent from a given process to any other process, and the `recv_counts` array tells the function how many elements come from each process to a given process. However, to determine where the elements of a sublist going to a given process begin, `MPI_Alltoallv` also needs an offset or **displacement** for the first element that goes from the process to each process, and the first element that comes to a process from each process. In our setting, these displacements can be obtained from the exclusive prefix sums of `send_counts`. Thus, in our example, the `send_displs` arrays on the different processes are

```
Process 0 : {0, 1, 4}
Process 1 : {0, 2, 2}
Process 2 : {0, 0, 3}
```

and the `recv_displs` arrays are

```
Process 0 : {0, 1, 3}
Process 1 : {0, 3, 3}
Process 2 : {0, 0, 2}
```

Note also that by adding in the final elements of `recv_counts` to the corresponding final elements `recv_displs` we get the total number of elements needed for `recv_data`:

```
Process 0 : recv_data[], 3 + 0 ints
Process 1 : recv_data[], 3 + 3 ints
Process 2 : recv_data[], 2 + 1 ints
```

We'll just use a *large* n -element array for `recv_data`. So we won't need to worry about allocating the exact amount of needed storage for each process' `recv_data`. After the call to `MPI_Alltoallv`, each process sorts its local list.

Note that, in general, we'll need to first get the *numbers* of elements coming from each process. We could use `MPI_Alltoallv` to do this, but since each process is sending a single `int` to every process, we can use a somewhat simpler function: `MPI_Alltoall`:

```
int MPI_Alltoall(
    const void *send_data      /* in */,
    int send_count             /* in */,
    MPI_Datatype send_type     /* in */,
    void *recv_data           /* out */,
    int recv_count             /* in */,
    MPI_Datatype recv_type     /* in */,
    MPI_Comm comm             /* in */);
```

For us, each process should put in `send_data` the number of elements going to each process; `send_count` and `recv_count` should be 1, and the types should both be `MPI_INT`.

At this point our convention is that we need to gather the entire list onto process 0. At first, it might seem that we can simply use `MPI_Gather` to collect the distributed list onto process 0. However, `MPI_Gather` expects each process to contribute the same number of elements to the global list, and, in general, this won't be the case. We encountered another version of `MPI_Gather`, `MPI_Gatherv`, back in Chapter 3. (See Exercise 3.13.) This has the following syntax:

```
int MPI_Gatherv(
    void*          loc_list           /* in */,
    int            loc_list_count     /* in */,
    MPI_Datatype    send_elt_type     /* in */,
    void*          list               /* out */,
    const int*      recv_counts       /* in */,
    const int*      recv_displs       /* in */,
    MPI_Datatype    recv_elt_type     /* in */,
    int            root               /* in */,
    MPI_Comm        comm              /* in */);
```

This is very similar to the syntax for `MPI_Gather`:

```
int MPI_Gather(
    void*          loc_list           /* in */,
    int            loc_list_count     /* in */,
    MPI_Datatype    send_elt_type     /* in */,
    void*          list               /* out */,
    const int       recv_count        /* in */,
    MPI_Datatype    recv_elt_type     /* in */,
    int            root               /* in */,
    MPI_Comm        comm              /* in */);
```

The only differences are that in `MPI_Gatherv`, there is an *array* of `recv_counts` instead of a single `int`, and there is an array of displacements. These differences are completely natural: each process is contributing a collection `loc_list` of `loc_list_count`, each of which has type `send_elt_type`. In `MPI_Gather`, each process contributes the same number of elements, so `loc_list_count` is the same for each process. In `MPI_Gatherv`, the processes contributed different numbers of elements. So instead of a single `recv_count`, we need an array of `recv_counts`, one for each process. Also, by analogy with `MPI_Alltoallv`, we need an array of *displacements*, `recv_displs`, that indicates where in `list` each process' contribution begins.

So we need to know how many elements each process is contributing, and we need to know the offset in `list` of the first element of each process' contribution.

Now recall that each process has built two arrays that are used in the call to `MPI_Alltoallv`. The first one specifies the number of elements the process receives from every process, and the second one specifies the offset (or displacement) of the first element received from each process. If we call these arrays `my_fr_counts` and `my_fr_offsets`, respectively, we can get the total number of elements received from each process by adding the last elements

```
my_new_count = my_fr_counts[p-1] + my_fr_offsets[p-1];
```

We can “gather” these counts onto process 0 to get the elements of `recv_counts`, and we can form the exclusive prefix sums of `recv_counts` to get `recv_displs`.

To summarize, then, Program 7.5 shows pseudocode for this version of sample sort.

```

/* s = global sample size */
loc_s = s/p;
Gen_sample(my_rank, loc_list, loc_n, loc_samp, loc_s);
Gather_to_0(loc_samp, global_sample);
if (my_rank == 0) FindSplitters(global_sample, splitters);
Broadcast_from_0(splitters);

/* Each process has all the splitters */
Sort(loc_list, loc_n);
/* my_to_counts has storage for p ints */
Count_elts_going_to_procs(
    loc_list, splitters, my_to_counts);

/* Get number of elements coming from each process
   *      in my_fr_counts */
MPI_Alltoall(my_to_counts, 1, MPI_INT,
    my_fr_counts, 1, MPI_INT, comm);

/* Construct displs or offset arrays for each process */
Excl_prefix_sums(my_to_counts, my_to_offsets, p);
Excl_prefix_sums(my_fr_counts, my_fr_offsets, p);

/* Redistribute the elements of the list */
/* Each process receives its elements into tlist */
MPI_Alltoallv(
    loc_list, my_to_counts, my_to_offsets, MPI_INT,
    tlist, my_fr_counts, my_fr_offsets, MPI_INT, comm);
my_new_count = my_fr_offsets[p-1] + my_fr_counts[p-1];

Sort(tlist, my_new_count);

/* Gather tlists to process 0 */
MPI_Gather(&my_new_count, 1, MPI_INT, bkt_counts, 1,
    MPI_INT, 0, comm);
if (my_rank == 0)
    Excl_prefix_sums(bkt_counts, bkt_offsets, p);
MPI_Gatherv(tlist, my_new_count, MPI_INT, list, bkt_counts,
    bkt_offsets, MPI_INT, 0, comm);

```

Program 7.5: Pseudocode for first MPI implementation of sample sort.

Table 7.14 Run-times of first MPI implementation of sample sort (times are in seconds).

Processes	Run-time
1	9.94e-1
2	4.77e-1
4	2.53e-1
8	1.32e-1
16	7.04e-2
32	5.98e-2
qsort	7.39e-1

Table 7.14 shows run-times of this implementation in seconds. The input list has $2^{22} = 4,194,304$ ints, and the sample has 16,384 ints. The last row of the table shows the run-time of the C library `qsort` function. We’re using a system running Linux with two Xeon Silvers 4116 running at 2.1 GHz. Each of the Xeons has 24 cores, and the MPI implementation is MPICH 3.0.4. Compiler optimization made little difference in overall run-times. So we’re reporting run-times with no optimization.

Second implementation

Our first MPI implementation seems to be fairly good. Except for the 32 core run, the efficiency was at least 0.88, and with 2 or more cores, it is always faster than the C library `qsort` function. So we would like to improve the efficiency when we’re using 32 cores.

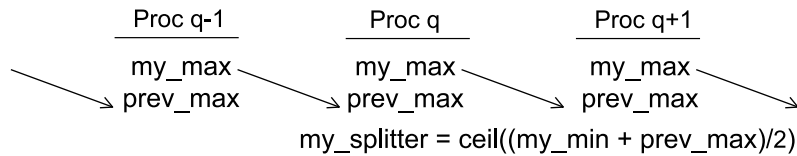
There are several modifications we can make to the code that may improve performance:

1. Replace the serial sort of the sample with a parallel sort.
2. Use a parallel algorithm to find the splitters.
3. Write our own implementation of the `Alltoallv` function.

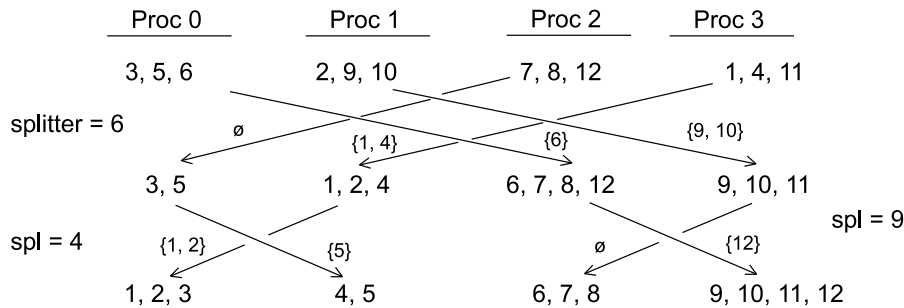
The first two modifications are straightforward. For the first, we can replace the gather of the sample to process zero and the serial sort of the sample, by a parallel odd-even transposition sort. (See Section 3.7.)

After the parallel sort of the sample, the sample will be distributed among the processes. So there’s no need to scatter the sample, and the keys on process $q - 1$ are all $\leq$ the keys on process q , for $q = 1, 2, \dots, p - 1$. Thus, since there are p splitters, we can compute `splitter[i]` by sending the maximum element of the sample on each process to the next higher-ranked process (if it exists), and the next higher-ranked process can average this value with its minimum value. (See Fig. 7.9.)

So let’s take a look at implementing `Alltoallv`. When p is a power of 2, we can implement this with a butterfly structure (Fig. 3.9). First we’ll use a small example to

**FIGURE 7.9**

Finding the splitters.

**FIGURE 7.10**

Butterfly implementation of Alltoallv.

illustrate how this is done. We'll then talk about the general case. So suppose we have four processes, and the splitters are $\{-\infty, 4, 6, 9, \infty\}$. Also suppose that the sublists are shown in the following table:

	Processes			
	0	1	2	3
sublists	3, 5, 6	2, 9, 10	7, 8, 12	1, 4, 11

Then a butterfly implementation of Alltoallv can be illustrated as shown in Fig. 7.10.

The algorithm begins with the sorted sublists distributed among the processors. The first stage uses the “middle” splitter, $\text{splitter}[2] = 6$ to divide the keys. Keys that are less than 6 will go to the first half of the distributed list, and keys greater than or equal to 6 will go to the second half of the distributed list. Processes 0 and 2 exchange keys, and processes 1 and 3 exchange keys. After the exchanges take place what remains of the old sublist is merged with the received keys.

Now that the distributed list has been split into two halves, there is no need for further communication between the processes that are less than $p/2$ and the processes that are greater than or equal to $p/2$. So we can now split the first half of the list between processes 0 and 1, and we can split the second half of the list between processes 2 and 3.

To split the first half of the list among processes 0 and 1, we'll use `splitter[1] = 4`, and to split the second half, we'll use `splitter[3] = 9`. As in the first “phase,” the paired processes (0–1 and 2–3) will exchange keys, and each process will merge its remaining keys with its new keys.

This gives us a distributed, sorted list, and we can use `MPI_Gather` and `MPI_Gatherv` to gather the list to process 0: this is the same as the final step in the algorithm we used for the first MPI implementation.

So let's take a look at some of the details in the implementation of `Alltoallv`. (See Program 7.6.)

```

1  unsigned bitmask = which_splitter = p >> 1;
2
3  while (bitmask >= 1) {
4      partner = my_rank ^ bitmask;
5      if (my_rank < partner) {
6          Get_send_args(loc_list, loc_n, &offset, &count,
7                      splitter[which_splitter], SEND_TO_UPPER);
8          new_loc_n = offset;
9          bitmask >>= 1;
10         which_splitter -= bitmask /* splitter for next pass */
11     } else {
12         Get_send_args(loc_list, loc_n, &offset, &count,
13                     splitter[which_splitter], SEND_TO_LOWER);
14         new_loc_n = loc_n - count;
15         bitmask >>= 1;
16         which_splitter += bitmask;
17     }
18     MPI_Sendrecv(loc_list + offset, count, MPI_INT, partner,
19                 0, rcv_buf, n, MPI_INT, partner, 0, comm, &status);
20     MPI_Get_count(&status, MPI_INT, &rcv_count);
21
22     loc_n = new_loc_n;
23     if (my_rank < partner) {
24         offset = 0;
25     } else {
26         offset = count;
27     }
28
29     /* Merges loc_list and rcv_buf into tmp_buf and swaps
30      * loc_list and tmp_buf */
31     Merge(&loc_list, &loc_n, offset,
32          rcv_buf, rcv_count, &tmp_buf);
33 } /* while bitmask >= 1 */

```

Program 7.6: Pseudocode for the butterfly algorithm in the second MPI sample sort.

The key to understanding how it works is to look at the binary or base 2 representations of the various scalar values. The variable `bitmask` determines which processes are paired: process `my_rank` is paired with the process with rank

```
partner = my_rank ^ bitmask;
```

(Recall that `^` takes the bitwise exclusive or of its operands.) The initial value of `bitmask` is `p >> 1`, and after each phase of the butterfly, it's updated by a right shift `bitmask >= 1`. When it's equal to 0, the iterations stop.

In our example, when $p = 4 = 100_2$, `bitmask` will take on the values 10_2 , 01_2 , and 00_2 . The following table shows which processes are paired:

bitmask	Processes			
	0 = 00_2	1 = 01_2	2 = 10_2	3 = 11_2
10_2	2 = 10_2	3 = 11_2	0 = 00_2	0 = 01_2
01_2	1 = 01_2	0 = 00_2	3 = 11_2	2 = 10_2

Note that all the processes will have the same value of `bitmask`.

The variable `which_splitter` is the subscript of the element in the `splitter` array that's used to determine which elements should be sent from one process to its partner. So, in general, the processes will have different values of `which_splitter`. It is also initialized to `p >> 1`, but it is updated using its value in the previous iteration and the updated `bitmask`. If the process' current partner has a higher rank, then, in the next iteration, it should choose the splitter

```
which_splitter = which_splitter - bitmask
```

If the partner has a lower rank, it should choose the splitter

```
which_splitter = which_splitter + bitmask
```

The following table shows the values of `which_splitter` for each process when $p = 8$. (`bitm` is an abbreviation of `bitmask`.)

New bitm	Processes							
	0 = 000_2	1 = 001_2	2 = 010_2	3 = 011_2	4 = 100_2	5 = 101_2	6 = 110_2	7 = 111_2
XXX	4 = 100_2	4 = 100_2	4 = 100_2	4 = 100_2	4 = 100_2	4 = 100_2	4 = 100_2	4 = 100_2
010 ₂	2 = 010_2	2 = 010_2	2 = 010_2	2 = 010_2	6 = 110_2	6 = 110_2	6 = 110_2	6 = 110_2
001 ₂	1 = 001_2	1 = 001_2	3 = 011_2	3 = 011_2	5 = 101_2	5 = 101_2	7 = 111_2	7 = 111_2

The function `Get_send_args` determines the arguments `offset` and `count` that are used in the call to `MPI_Sendrecv`. The variable `offset` gives the subscript of the first element in `loc_list` that will be sent to the partner, and the variable `count` is the number of elements that will be sent. They are determined by a search of the local list for the location of the splitter. This can be carried out with a linear or a binary search. We used linear search.

After the call to `MPI_Sendrecv`, we use the `status` argument and a call to `MPI_Get_count` to determine the number of elements that were received, and then

call a merge function that merges the old local list and the received list into `tmp_buf`. At the end of the merge, the new list is swapped with the old `loc_list`.

An important difference between the implementation that uses `MPI_Alltoallv` and this implementation is that to use `MPI_Alltoallv`, we must determine *before* the call

- How many elements go *from* each process *to* every other process, and
- The offsets or displacements of the first element going from each process to every other process.

As we've already seen, this requires communication, so it's relatively time consuming. With the hand-coded implementation, we can determine these values as the algorithm proceeds, and the *amount* of the data is, effectively, included in the communication of the actual data. We can access the amount by using the status and the (local) MPI function `MPI_Get_count` after receiving the data.

In the approach outlined above for implementing the data exchange, we use `MPI_Sendrecv` and a receive buffer that's large enough to receive any list (e.g., an n -element buffer). A more memory-efficient implementation can split the send and the receive, and before the receive is executed, it can call `MPI_Probe`:

```
int MPI_Probe(
    int          source      /* in */,
    int          tag         /* in */,
    MPI_Comm     comm        /* in */,
    MPI_Status*  status_p    /* out */);
```

This function will block until it is notified that there is a message from process `source` with the tag `tag` that has been sent with communicator `comm`. When the function returns, the fields referred to by `status_p` will be initialized, and the calling process can use `MPI_Get_count` to determine the size of the incoming message. With this information, the process can, if necessary, call `realloc` to enlarge the receive buffer before calling `MPI_Recv`.

To ensure safety in the MPI sense (see Section 3.7.3) of this pair of communications, we can use a **nonblocking** send instead of the standard send. The syntax is

```
int MPI_Isend(
    void*        buf         /* in */,
    int          count       /* in */,
    MPI_Datatype datatype    /* in */,
    int          dest        /* in */,
    int          tag         /* in */,
    MPI_Comm     comm        /* in */,
    MPI_Request* request_p   /* out */);
```

The first six arguments are the same as the arguments to `MPI_Send`. However, unlike `MPI_Send`, `MPI_Isend` only *starts* the communication. In other words, it notifies the system that it should send a message to `dest`. After notifying the system, `MPI_Isend`

Table 7.15 Run-times of the two MPI implementation of sample sort (times are in seconds).

Processes	Version 1 Run-time	Version 2 Run-time
1	9.94e-1	8.39e-1
2	4.77e-1	4.03e-1
4	2.53e-1	2.10e-1
8	1.32e-1	1.10e-1
16	7.04e-2	6.36e-2
32	5.98e-2	4.67e-2
qsort	7.39e-1	7.39e-1

returns: the “I” stands for “immediate,” since the function returns (more or less) immediately. Because of this, the user code *cannot* modify the send buffer `buf` until after the communication has been explicitly *completed*.

There are several MPI functions that can be used for completing a nonblocking operation, but the only one we’ll need is `MPI_Wait`:

```
int MPI_Wait(
    MPI_Request* request_p /* in */,
    MPI_Status* status_p /* out */);
```

The `request_p` argument is the argument that was returned by the call to `MPI_Isend`. It is an opaque object that is used by the system to identify the operation. When our program calls `MPI_Wait`, the program blocks until the send is completed. For this reason, nonblocking communications are sometimes called “split” or “two-phase” communications. We won’t make use of the `status_p` argument, so we can pass in `MPI_STATUS_IGNORE`.

To summarize, if we want to minimize memory use, we can replace the call to `MPI_Sendrecv` in Program 7.6 with the function shown in Program 7.7. The function

```
int MPI_Abort(
    MPI_Comm comm /* in */,
    int errorcode /* in */);
```

will terminate all the processes in `comm` and return `errorcode` to the invoking environment. So we need to define the constant `REALLOC_FAILED`. Note that current implementations of MPI will terminate *all* processes, not just those in `comm`.

Let’s take a look at the performance of our the version of sample sort that uses a hand-coded butterfly implementation of `Alltoallv` and `MPI_Sendrecv` (with large buffers). Table 7.15 shows run-times of this version and our original version of sample sort that uses `MPI_Alltoallv`. All the times were taken on the same system (see page 423). Both versions used no compiler optimization, and the input data was identical. We see that the second version is always faster than the first, and it is usually more efficient. The sole exception is that with $p = 16$, the first version has an ef-

```

1 void Send_rcv(int snd_buf[], int count, int** rcv_buf_p,
2             int* rcv_buf_sz_p, int partner, MPI_Comm comm) {
3     int rcv_count, *rcv_ptr;
4     MPI_Status status;
5     MPI_Request req;
6
7     /* Start the send */
8     MPI_Isend(snd_buf, count, MPI_INT, partner, 0, comm, &req);
9
10    /* Wait for info on storage needed for rcv_buf */
11    MPI_Probe(partner, 0, comm, &status);
12    MPI_Get_count(&status, MPI_INT, &rcv_count);
13    if (rcv_count > *rcv_buf_sz_p) {
14        rcv_ptr = realloc(*rcv_buf_p, rcv_count);
15        if (rcv_ptr == NULL)
16            /* Call to realloc failed, quit */
17            MPI_Abort(comm, REALLOC_FAILED);
18        else
19            *rcv_buf_p = rcv_ptr;
20    }
21
22    /* Now do the receive */
23    *rcv_buf_sz = rcv_count;
24    MPI_Recv(*rcv_buf_p, rcv_count, MPI_INT,
25            partner, 0, comm, &status);
26
27    /* Complete the send */
28    MPI_Wait(&req, MPI_STATUS_IGNORE);
29 } /* Send_rcv */

```

Program 7.7: Send_rcv function for minimizing memory requirements.

iciency of 0.88, while the second has an efficiency of 0.82. This is because the run-time of the first version is so much greater. Note that neither version has a very high efficiency with $p = 32$: the first version has an efficiency of 0.52 and the second 0.56. So we'll look at a couple of further improvements to the second version in Exercise 7.22.

7.2.9 Implementing sample sort with CUDA

As usual, we'll look at two implementations. The first splits the implementation between the host and the device. The second has the same basic structure as our second serial implementation. Unlike the first implementation, however, the entire algorithm, except for synchronization, is executed on the device.

First implementation

For the first implementation, we'll avoid issues with interblock synchronization by restricting ourselves to a single thread block. We'll start the implementation by using the host to choose the sample, sort the sample, and choose the splitters. Then a kernel will build the data structures needed to assign elements to the correct buckets. As with the MIMD implementations, each thread will do the work associated with one sublist and one bucket.

The host code is identical to the first serial implementation: it uses a random number generator to choose the sample, the `qsort` library function to sort the sample, and a simple `for` loop to choose the splitters, which are computed from equally spaced elements of the sample.

The device code starts by finding out the number of elements from the list that will be mapped to each bucket. These numbers are stored in an array `counts`. Then we implement parallel exclusive prefix sums of the elements of `counts`. This will tell us the subscript of the first element of each thread's bucket. When we have the prefix sums, each thread can copy the elements of its bucket to contiguous locations in a temporary array. The device code finishes with each thread sorting its bucket and copying the elements of the bucket back to the original list. (See Program 7.8.)

Let's take a look at some of the details. First, the list being sorted is the argument `list`, so it has n elements. As in our other examples, `splitters` has $b + 1$ or `th_per_blk + 1` elements, since we explicitly store `MINUS_INF` and `INF` as the first and last elements, respectively. The `counts` array also has storage for `th_per_blk + 1` elements. This will allow us to find the first and last elements in a bucket without treating the last bucket as a special case.

We'll use the shared memory array `shmem` to store and sort the individual buckets. It will be partitioned into subarrays: the i th subarray will store the i th bucket, for $i = 0, 1, \dots, b - 1$. This will limit the number of elements we can sort. For example, in our system, the maximum amount of shared memory available to a thread block is 49,152 bytes or $49,152/4 = 12,288$ ints.

Note that after most of the function calls, we need a barrier (i.e., a call to `__syncthreads`), because a subsequent function call will have a thread that is making use of the output from the earlier call that may have been computed by another thread. For example, the output of `Find_counts` is the array `counts`, and `counts[i]` is computed by the thread with `my_rank = i`, for $0 \leq i \leq \text{th_per_blk}$. But in the call to `Excl_prefix_sums`, a different thread will, in general, make use of this entry. So there would be a race condition if there weren't a barrier between the two calls.

Except for `Excl_prefix_sums`, the device functions called in the kernel are pretty straightforward to implement. In both `Find_count` and `Copy_to_my_bkt`, each thread iterates through all the elements of `list`. In `Find_counts`, each thread counts the number of elements belonging to its bucket. This information is used to determine both the locations and sizes of the buckets. In `Copy_to_my_bkt`, each thread copies the elements belonging to its bucket into the appropriate subarray of `shmem`.

```

1  __global__ void Dev_ssort(int list[], int splitters[],
2      int counts[], int n) {
3      int my_rank = threadIdx.x;  // Only one thread block
4      int th_per_blk = blockDim.x;
5      int my_first, my_last, my_count, *my_bkt;
6      __shared__ int shmem[MAX_SHMEM];  // n <= MAX_SHMEM
7
8      Find_counts(list, n, splitters, counts,
9          th_per_blk, my_rank);
10     __syncthreads();
11
12     my_count = counts[my_rank];
13     // Now counts has number of elements going to each bucket
14     // counts has th_per_blk+1 elements.
15     // Last element is uninitialized
16     Excl_prefix_sums(counts, th_per_blk+1, my_rank);
17     if (my_rank == 0) counts[th_per_blk] = n;
18     __syncthreads();
19
20     int* my_bkt = shmem + counts[my_rank];
21     Copy_to_my_bkt(list, n, splitters,
22         my_bkt, my_count, my_rank);
23
24     Serial_sort(my_bkt, my_count);
25     __syncthreads();
26
27     my_first = counts[my_rank];
28     my_last = counts[my_rank+1];
29     Copy_fr_my_bkt(list, my_bkt, my_rank, counts[my_rank],
30         counts[my_rank+1]);
31 }  /* Dev_ssort */

```

Program 7.8: Kernel code for the first CUDA implementation of sample sort.

The `Serial_sort` function can be any single-threaded sort. We used heap sort. Finally, in `Copy_fr_my_bkt`, each thread copies its sorted bucket back to the appropriate subarray of `list`.

Prefix sums

Recall that when we were discussing implementations of serial sample sort, we talked about prefix sums. (See paragraph ‘Prefix sums’ on p. 404.) To complete our implementation of this sample sort, we can use *exclusive* prefix sums of the elements of the `counts` array to determine *where* each bucket should be located in the `shmem` array. Recall that if x is a b -element array, then the exclusive prefix sums of x are the sums

$$0, x[0], x[0]+x[1], \dots, x[0] + x[1] + \dots + x[b-2].$$

So a serial algorithm for computing exclusive prefix sums is

```
ps[0] = 0;
for (i = 1; i < b; i++)
    ps[i] = ps[i-1] + x[i-1];
```

As in the serial implementation, the exclusive prefix sums tell us where each bucket begins. This can be done “in place,” i.e., the prefix sums are stored in `x` with the following algorithm:

```
told = x[0];
x[0] = 0;
for (i = 1; i < b; i++) {
    tnew = x[i];
    x[i] = told + x[i-1];
    told = tnew;
}
```

Since we were originally interested in serial prefix sums, this was all we needed. In fact, given the dependence of the calculation of the i th prefix sum on the $i - 1$ st, it might at first seem that this is inherently serial. However, there is a relatively simple parallel algorithm that is also a good deal faster than the serial algorithm:

```
if (my_rank == 0)
    tmp = my_rank;
else
    tmp = x[my_rank - 1];
__syncthreads();
x[my_rank] = tmp;
__syncthreads();

for (shift = 1; shift < b; shift <= 1) {
    if (my_rank >= shift) tmp = x[my_rank - shift];
    __syncthreads();
    if (my_rank >= shift) x[my_rank] += tmp;
    __syncthreads();
}
```

For example, suppose `x` has six elements that are initialized as shown in Table 7.16. Note that since the thread block can have multiple warps, we need the calls to `__syncthreads`. Without them one thread can “race ahead” of the other threads and use results from an earlier iteration.

This is not the most efficient parallel implementation, but it’s easy to understand and code. See, for example, [27] for a more efficient implementation.

Performance

How does our CUDA sample sort perform? Table 7.17 shows results taken on an Nvidia system with an Intel Xeon Silver 4116 host running at 2.1 GHz. The devices are Pascal GPUs with a maximum clock rate of 1.73 GHz, 20 SMs, and 128 SPs

Table 7.16 Computation of exclusive prefix sums using the parallel algorithm.

Time	Subscripts in x					
	0	1	2	3	4	5
Start	3	6	7	5	3	5
Initialization	0	3	6	7	5	3
shift = 1	0	3	9	13	12	8
shift = 2	0	3	9	16	21	21
shift = 4	0	3	9	16	21	24

Table 7.17 First version of CUDA sample sort, with $n = 12,288$, $s = 1024$. Times are in seconds.

Threads	Run-times
32	7.48e-3
64	6.25e-3
128	5.14e-3
256	5.18e-3
512	5.19e-3
1024	5.34e-3
qsort	2.69e-3

per SM. For the performance tests, $n = 12,288$, $s = 1024$, the sample sort was run at least 30 times with each set of input, and minimum run-times are reported. The briefest examination shows that the performance is *not* very good. The C library `qsort` function running on a single core of the host is almost twice as fast as the fastest run of the sample sort. Also, since the program is *slower* with 512 and 1024 threads than it is with 256, the program isn't at all scalable.

Table 7.18 shows the fastest run-times of the program for a given sample size. For each sample size, we ran the program with the number of threads ranging from 64 up to the sample size, except for sample size 2048, where we only ran up to 1024 threads, since the maximum number of threads that can be run on a single block in our system is 1024. These data show that for larger sample sizes, the host code (choosing the sample, sorting it, and finding the splitters) formed a huge part of the overall run-time. It seems clear that if the sample size is large relative to the list size, then our method for choosing the sample is *far* too slow. The data also show that the time spent on the device decreases as the sample size increases. There two possible reasons for this: as the sample size increases, on average, the amount of work per thread decreases. Also, as the sample size increases, it's probable that the buckets will become more uniform in size, and the total work per thread will decrease.

Finally, observe that when two SIMD threads run a serial sorting algorithm, such as heap sort, on two different lists, we would expect that there would be large amounts

Table 7.18 First version of CUDA sample sort. Fastest times for various sample sizes. Times are in milliseconds.

Sample Size	Number of Threads	Host Time	Device Time	Total time
128	128	0.00572	3.29	3.30
256	256	0.176	3.02	3.20
512	512	0.614	2.95	3.56
1024	256	2.93	2.25	5.18
2048	512	8.93	1.27	10.2

Table 7.19 Run-times for 32 CUDA threads running heap sort. Times are in milliseconds.

Number of Thread Blocks	Run-time
1	1.16
2	1.05
4	1.04
8	1.02
16	0.814
32	0.764
qsort	1.82

of thread divergence, and hence very little parallelism. We looked at this by timing a CUDA program in which a list with n elements is shared among a collection of threads, and each thread must sort its sublist using heap sort. We sorted each list using 32 CUDA threads. However, we varied the number of thread blocks. When there is a single block of 32 threads, all the threads are in the same warp, and hence they are SIMD threads. At the other extreme, each thread belonged to its own thread block, and to all intents and purposes, they are MIMD threads. Table 7.19 shows the results when $n = 12,288$, and the program is run on the same Pascal system used for the sample sort. The C library function `qsort` was run on a core of the host system on the same list. So we see that even with 32 SIMD threads, the CUDA heap sort is faster than the C library `qsort`. Of more interest is that the speedup of 32 “MIMD” threads—i.e., 1 thread/block—is better than 1.5. So for this problem, thread divergence increased the run-time by more than 50%.

We’ll make use of our observations on the performance of the first CUDA sample sort in the design of our second CUDA implementation.

A second CUDA implementation of sample sort

It is easy to see how we might make a number of improvements to the first CUDA implementation. For example, the data in Table 7.18 show that for large sample sizes, the overall run-time of the program is dominated by the time spent in the serial code:

choosing the sample, sorting it, and finding the splitters. Clearly, then, we should use algorithms that parallelize these steps.

Another obvious issue is that the first implementation can't sort a list with more than 12,288 elements, and it may be very difficult to outperform a good serial code on lists this small. So we should write a program that will sort relatively large lists, and to bring the full power of the GPU to bear, we should write a program that makes good use of many, large thread blocks.

A final issue with the first implementation is that the threads aren't making use of the power of SIMD execution. When each thread is running a complex algorithm with many conditional branches—e.g., an algorithm like heap sort—we have little reason to think that the threads will be executing the same instruction at the same time. On a MIMD system, this is generally not a problem, but the ideal SIMD application has all of the threads executing the same instruction simultaneously. When threads need to take different branches, other threads are necessarily idled, and their computational power is wasted.

Several researchers have developed implementations of sample sort for GPUs. Our solution is a simplified version of the implementation discussed in [13]. In this implementation, each bucket corresponds to a thread block. Here's an outline; we'll go into more detail shortly.

1. **Choose the sample.** Each thread block operates independently in choosing the sample. First each block's sublist is copied into shared memory and sorted using a slightly modified version of our first bitonic sort algorithm (see Exercise 6.12). Then threads choose equally spaced elements from the sorted sublist. These are copied into a `sample` array in global memory.
2. **Sort the sample.** The global sample is sorted using our second bitonic sort algorithm (see Section 6.14.4).
3. **Choose the splitters.** A collection of threads chooses equally spaced splitters from `sample`.
4. **Determine destination buckets.** For each element of the list, determine its destination bucket by using a binary search of the splitters. The number of elements going from sublist j to bucket i is stored in `mat_counts[i][j]`. After determining the destination bucket of an element, we use `atomicAdd` to increment the appropriate entry in `mat_counts`.
5. **Parallel prefix sums.** Form the exclusive prefix sums of the rows of `mat_counts`. After the completion of the prefix sums, `mat_counts[i][j]` will be the index in sublist j of its first element going to bucket i . This step also determines the total number of elements going to each bucket i , which is the sum of the elements in row i of the original `mat_counts`.
6. **Serial prefix sums.** To determine the location of the start of each bucket in a temporary array, we form the exclusive prefix sums of the bucket sizes. Unlike the preceding step, this is just a serial calculation.
7. **Send elements to buckets.** Now we map the elements of the list to their destination buckets in a temporary array.

- 8. Sort buckets.** To finish up, we want to sort each bucket. However, to use the single block bitonic sort, each bucket needs to store 2^k elements, for some nonnegative integer k . To arrange this, we copy each bucket into temporary storage, pad it with copies of ∞ , sort the padded list, and copy the elements of the sorted, padded list that are $< \infty$, back into the original list.

We specify `blk_ct`, `th_per_blk`, the list size `n`, and the sample size `s`, on the command-line. We make the following assumptions:

- All four of the command-line arguments are powers of 2.
- The number of elements in the list $n = 2 * \text{blk_ct} * \text{th_per_blk}$. This is the assumption that's made on the relation between these values in our implementation of bitonic sort (see Section 6.14.4).
- $s < n$. If $s = n$, there's no point in using sample sort: we may as well use bitonic sort for the entire list.
- $\text{blk_ct} \leq s$. The number of splitters is the same as the number of buckets, which is the same as the number of thread blocks. Since we choose the splitters from the sample, and we need one splitter for each thread block, we can't have more splitters than sample elements.

Note that except for the serial prefix sums (Step 6), all of the steps are parallel, but unlike our other CUDA programs, the different steps may use differing numbers of thread blocks and differing numbers of threads per block.

Let's take a look at some of the details.

- 1. Choose the sample.** The number of blocks and the number of threads per block are the same as the command-line arguments.
- 2. Sort the sample.** Since we're using our bitonic sort to sort the sample, we want

$$s = 2 \times \text{blk_ct} \times \text{th_per_blk}$$

So the number of threads per block in this kernel is

$$\text{sth_per_blk} = \min\{s/2, \text{th_per_blk}\},$$

and the number of blocks is

$$\text{sblk_ct} = \frac{s}{2 \times \text{sth_per_blk}}.$$

- 3. Choose the splitters.** We use one thread to compute one splitter by simply finding the gap between splitters and multiplying by the thread's global rank. This is ideal for SIMD parallelism. So we use as many threads per block as possible. This is a constant defined in the program, `MAX_TH_PER_BLK`. (On current generation systems, it's 1024.) In case there are more splitters than this, we define the number of blocks to be

$$\text{spl_blk_ct} = \text{ceil}(\text{blk_ct} / \text{MAX_TH_PER_BLK})$$

4. **Determine destination buckets.** This uses the `blk_ct` blocks and `th_per_blk` threads per block, so each thread determines the destination buckets of two elements of the list.
5. **Parallel prefix sums.** Each bucket requires a prefix sum, and each row has `blk_ct` elements. So, in general, we want one thread block for each row, and `blk_ct` threads per row to carry out a parallel prefix sum. However, if there are more blocks than `MAX_TH_PER_BLK`, then the number of threads per block must be set to `MAX_TH_PER_BLK` instead of `blk_ct`.
The algorithm is a modified version of an efficient prefix sum described in [27]. Note that we want to compute the subscript of the first element in each sublist that's being mapped to a given bucket. So using rows that correspond to a bucket, rather than the transposed array with rows that correspond to a sublist, results in a more efficient use of cache: we are using contiguous entries in a row rather than strided elements in a column.
6. **Serial prefix sums.** If `blk_ct` $\leq$ `MAX_TH_PER_BLK`, this could be implemented with a single thread block, but in the interests of simplifying the program we opted for a serial prefix sum.
7. **Send elements to buckets.** This uses the command-line arguments `blk_ct` and `th_per_blk`, so each thread is responsible for two elements. Given the k th element of the current sublist, we determine the subscript of the first element in the sublist that maps to the same bucket, `first_elt_sub`. The difference

$$k - \text{first_elt_sub}$$

gives the offset of the element into the destination bucket. The previously calculated prefix sums allow us to determine the subscript of the first element in the destination bucket. Adding the offset and the start of the bucket gives the subscript of the destination of the element.

8. **Sort buckets.** Here, we want to use a single thread block to sort each bucket using the single-block implementation of bitonic sort. However, we expect that in general the buckets will have more than $2 \times \text{th_per_blk}$ elements. So we may be able to make use of large thread blocks. When this is the case, we use `MAX_TH_PER_BLK` instead of the command line value `th_per_blk`. Note that the single thread block bitonic sort has been modified so that it can handle arbitrary powers of 2 for both the number of threads and the number of elements being sorted.

An additional issue is the possibility that the size of the list being sorted by a thread block is larger than the maximum available storage for shared memory. This depends both on the size of the bucket being stored in the list and the padding that may be required to adjust the list size so that it's a power of two. So before calling the kernel that sorts each thread block, we find the padded size of each bucket and allocate storage in global memory for these buckets. Before the kernel calls the bitonic sort device function, it checks whether it needs to use global memory, and if it does, the padded bucket is copied into global memory. If it doesn't, the padded bucket is copied into shared memory.

Table 7.20 Average run-times and maximum bucket sizes for second CUDA sample sort. Times are in seconds.

blk_ct	th_per_blk	n	s	Run-time	qsort	Max Bkt Sz
256	256	131,072	16,384	1.58e-3	2.90e-2	2048
512	256	262,144	32,768	4.47e-3	5.49e-2	4096
256	512	262,144	32,768	2.14e-3	5.49e-2	2048
1024	256	524,288	65,536	7.30e-3	1.22e-1	8192
512	512	524,288	65,536	5.28e-3	1.22e-1	4096
256	1024	524,288	65,536	3.28e-3	1.22e-1	4096
1024	512	1,048,576	131,072	9.53e-3	2.46e-1	8192
512	1024	1,048,576	131,072	6.25e-3	2.46e-1	4096
1024	1024	2,097,152	262,144	1.42e-2	5.20e-1	8192

Performance of the second CUDA sample sort

We used the same Pascal-based system that we used for the first implementation. (See Section ‘Performance’ on p. 432.) Since there was considerable variation in the run-times for the individual executions, we’re reporting *average* run-times over 30 runs. The sample size didn’t seem to make a very large difference in the overall run-times, so we are only reporting on sample size = (list size)/8. (See Table 7.20.) The column labeled “qsort” is the run-time (in seconds) of the C library `qsort` function. It is run on the same list that the sample sort was run on, but it runs on a single core of the Intel Xeon host. The column labeled “Max Bkt Sz” is the size of the largest bucket after it has been padded with elements to make its size a power of two. For all of the problems in the table, the CUDA sample sort is more than 15 times faster than the `qsort` library functions. However, for very small problems ($n < 4096$), the `qsort` library function *was* faster. Sample sizes that were between 4 and 16 times smaller than the list size didn’t make much difference in the overall run-times. However, sample sizes that were much smaller than a factor of 16 less than the list size could make a large difference. For example, with 1024 blocks, 1024 threads per block, a list of size 2,097,152, and a sample size of 16,384, the program was 40% slower than the program with the same input, but a sample of size 262,144. For large lists, sample size also had a noticeable effect on the maximum bucket size. Fortunately, the use of global memory for temporary storage of the buckets didn’t seem to have a very significant effect on the overall run-time.

Perhaps, the most striking feature of the table is the decrease in run-times when the number of blocks was decreased, but the number of threads per block was increased. For example, with 262,144 threads and a list containing 524,288 ints, using 256 blocks of 1024 threads was more than twice as fast as using 1024 blocks of 256 threads. If we time the various kernels called by the sample sort, it’s clear that most of the difference is due to the increased cost of the kernel that builds the matrix of counts. With 256 blocks of 1024 threads, we need to compute $256 \times 256 = 65,536$ entries, while with 1024 blocks of 256 threads, we need to compute $1024 \times 1024 =$

1,048,576 entries, and there is relatively little difference in the average cost of computing an entry. (See Exercise 7.24.)

7.3 A word of caution

In developing our solutions to the n -body problem and the problem of sorting using sample sort, we chose our serial algorithms because they were easy to understand, and their parallelizations were relatively straightforward. In no case did we choose a serial algorithm because it was the fastest or because it could solve the largest problem. Thus, it should not be assumed that either the serial or the parallel solutions are the best available. For information on “state-of-the-art” algorithms, see the bibliography, especially [14] for the n -body problem and [2] and [22] for sorting.

7.4 Which API?

How can we decide which API, MPI, Pthreads, OpenMP, or CUDA is best for our application? In general, there are many factors to consider, and the answer may not be at all clear-cut. However, here are a few points to consider.

As a first step, decide whether the application is suitable for use with CUDA. If a serial solution involves using more or less the same operations on different data items, then CUDA is a very good candidate. On the other hand, if a serial solution involves lots of conditional branches that depend on the data, then CUDA may not be a good choice: you may have to rework the serial solution so that it’s more amenable to a SIMD parallelization.

As a second step, decide whether to use distributed-memory MIMD or shared-memory MIMD. To do this, first consider the amount of memory the application will need. In general, distributed-memory systems can provide considerably more main memory than shared-memory systems, so if the memory requirements are very large, you may need to use MPI.

If the problem will fit into the main memory of your shared-memory system, you may still want to consider using MPI. Since the total available cache on a distributed-memory system will probably be much greater than that available on a shared-memory system, it’s conceivable that a problem that requires lots of main memory accesses on a shared-memory system will mostly access cache on a distributed-memory system, and, consequently, have much better overall performance.

However, even if you’ll get a big performance improvement from the large aggregate cache on a distributed-memory system, if you already have a large and complex serial program, it often makes sense to write a shared-memory program. It’s often possible to reuse considerably more serial code in a shared-memory program than a distributed-memory program. It’s more likely that the serial data structures can be easily adapted to a shared-memory system. If this is the case, the development effort for the shared-memory program will probably be much less. This is especially

true for OpenMP programs, since some serial programs can be parallelized by simply inserting some OpenMP directives.

Another consideration is the communication requirements of the parallel algorithm. If the processes/threads do little communication, an MPI program should be fairly easy to develop, and very scalable. At the other extreme, if the processes/threads need to be very closely coordinated, a distributed-memory program will probably have problems scaling to large numbers of processes, and the performance of a shared-memory program should be better.

If you decided that shared-memory is preferable, you will need to think about the details of parallelizing the program. As we noted earlier, if you already have a large, complex serial program, you should see if it lends itself to OpenMP. For example, if large parts of the program can be parallelized with `parallel for` directives, OpenMP will be much easier to use than Pthreads. On the other hand, if the program involves complex synchronization among the threads—for example, read-write locks or threads waiting on signals from other threads—then Pthreads will be much easier to use.

7.5 Summary

In this chapter, we’ve looked at serial and parallel solutions to two very different problems: the n -body problem and sorting using sample sort. In each case we began by studying the problem and looking at serial algorithms. We continued by using Foster’s methodology for devising a parallel solution, and then, using the designs developed with Foster’s methodology, we implemented parallel solutions using Pthreads, OpenMP, MPI, and CUDA.

In developing the reduced MPI solution to the n -body problem, we determined that the “obvious” solution would be difficult to implement correctly, and it would require a huge amount of communication. We therefore turned to an alternative “ring pass” algorithm, which proved to be much easier to implement and is probably more scalable.

We considered implementing a CUDA n -body solver that used some of the same ideas we used in developing the reduced Pthreads and OpenMP solvers. However, we found that the amount of memory required would be too great. So we looked at an alternative approach to improving the performance of the basic CUDA n -body solver: we reduced the number of accesses to global memory by loading from global memory into fast, on-chip shared memory. Then, when we carried out the calculations, we accessed shared memory rather than global memory. We accomplished this by dividing the arrays into “virtual” tiles. Then each thread block loaded a tile, did all the required calculations involving the particles in that tile, and, when it was done with the particles in the current tile, proceeded to the next tile. Effectively, then, we used shared memory as a programmer-managed cache.

We looked at two serial implementations of sample sort. In the first, we chose the sample using a random number generator, sorted the sample, and chose “splitters” to

be equally spaced elements of the sample. Then the elements of the list were mapped to their destination “buckets” and each bucket was sorted.

The second serial implementation used a deterministic scheme to choose the sample, and the contents of the buckets were determined using *exclusive prefix sums*. An *inclusive* prefix sum of the elements of a list $\{a_0, a_1, \dots, a_{n-1}\}$ is the new list

$$a_0, a_0 + a_1, \dots, a_0 + a_1 + \dots + a_{n-1}.$$

An exclusive prefix sum starts with 0. So it is

$$0, a_0, a_0 + a_1, \dots, a_0 + a_1 + \dots + a_{n-2}.$$

We parallelized both implementations using data parallelism. For MIMD systems, the “sublists” were contiguous blocks of n/p elements of the original list and the “buckets” were contiguous blocks elements of the final list—which, in general, did not have exactly n/p elements. For CUDA, we used the same terminology, but the definitions were different. In the first implementation, we used a single thread block. So the sublists had $n/\text{th_per_blk}$ elements and the buckets had approximately $n/\text{th_per_blk}$ elements. In the second implementation, we used multiple thread blocks, and sublists had $n/\text{blk_ct}$ elements and buckets had approximately this many elements.

All of the implementations used similar ideas. In general, first implementations did the sampling, and identification of splitters using a single process/thread in the MIMD systems and the host processor with CUDA. The building of data structures, mapping of elements to buckets, and final sorting was done in parallel. The second implementation used parallelism at all stages, except that the OpenMP and Pthreads implementation used a serial sorting algorithm to sort the sample.

In closing, we looked briefly at the problem of deciding which API to use. Our first consideration was whether the problem was suitable for parallelization using CUDA: serial programs that use more or less the same operations on different data items are usually well suited to a CUDA parallelization. The second consideration is whether to use shared memory or distributed memory. To decide this, we should look at the memory requirements of the application and the amount of communication among the processes/threads. If the memory requirements are great or the distributed-memory version can work mainly with cache, then a distributed-memory program is likely to be much faster. On the other hand, if there is considerable communication, a shared-memory program will probably be faster.

When you’re choosing between OpenMP and Pthreads, if there’s an existing serial program and it can be parallelized by the insertion of OpenMP directives, then OpenMP is probably the clear choice. However, if complex thread synchronization is needed—for example, read-write locks or thread signaling—then Pthreads will be easier to use.

7.5.1 MPI

In the course of developing these programs, we learned about several additional MPI collective communication functions. In the first MPI implementation of sample sort, we recalled a generalization of `MPI_Gather`, `MPI_Gatherv`, which allows us to gather different numbers of elements from one process onto another:

```
int MPI_Gatherv(
    void*          loc_list          /* in */,
    int            loc_list_count    /* in */,
    MPI_Datatype    send_elt_type    /* in */,
    void*          list              /* out */,
    const int*      recv_counts       /* in */,
    const int*      recv_displs       /* in */,
    MPI_Datatype    recv_elt_type     /* in */,
    int             root              /* in */,
    MPI_Comm        comm              /* in */);
```

This gathers the contents of each process' `loc_list` into `list` on the process with rank `root` in `comm`. However, unlike `MPI_Gather`, the number of elements in `loc_list` may differ from one process to another. So the argument `recv_counts` tells us the number of elements each process is sending, and the argument `recv_displs` tells us where each process' first element goes in `list`.

We also used two MPI implementations of “all-to-all scatter-gather.” These combine the features of both scatter and gather: each process scatters its own collection to all the processes in the communicator, and each process gathers data from all the processes. The first implementation is `MPI_Alltoall`, which sends the same amount of data from each process to each process. The second implementation is `MPI_Alltoallv`, which can send different amounts of data to different processes. Their syntaxes are

```
int MPI_Alltoall(
    const void*     *send_data       /* in */,
    int             send_count       /* in */,
    MPI_Datatype     send_type        /* in */,
    void*           *recv_data       /* out */,
    int             recv_count       /* in */,
    MPI_Datatype     recv_type        /* in */,
    MPI_Comm        comm             /* in */);
```

```
int MPI_Alltoallv(
    const void*     *send_data       /* in */,
    const int*      *send_counts      /* in */,
    const int*      *send_displs     /* in */,
    MPI_Datatype     send_type        /* in */,
    void*           *recv_data       /* out */,
    const int*      *recv_counts      /* in */,
    const int*      *recv_displs     /* in */);
```

```

MPI_Datatype  recv_type      /* in */,
MPI_Comm      comm          /* in */);

```

We also learned about the nonblocking send function `MPI_Isend`:

```

int MPI_Isend(
    void*      buf           /* in */,
    int        count         /* in */,
    MPI_Datatype datatype    /* in */,
    int        dest          /* in */,
    int        tag           /* in */,
    MPI_Comm    comm         /* in */,
    MPI_Request* request_p   /* out */);

```

We used this to implement a memory-efficient send-receive. The first six arguments are the same as the arguments to `MPI_Send`. When it's called, it *starts* a send, but when it returns, the send will not, in general, be completed. The output argument, `request_p`, is an opaque object. It's used by the program and MPI system to identify the communication. When we want to finish the communication, we can call the function `MPI_Wait`:

```

int MPI_Wait(
    MPI_Request* request_p /* in */,
    MPI_Status*  status_p  /* out */);

```

When this function is called, it will block until the call associated with `request_p` (in our case, the call to `MPI_Send`) has completed. The `status_p` argument will return information on the call. In our setting we didn't use it. So we just passed in `MPI_STATUS_IGNORE`.

To determine whether the process with which we were communicating had sent its data, we used the function `MPI_Probe`.

```

int MPI_Probe(
    int        source        /* in */,
    int        tag           /* in */,
    MPI_Comm    comm         /* in */,
    MPI_Status* status_p     /* out */);

```

This blocks until it is notified that there is a message from process `source` with tag `tag` in communicator `comm`. The `status_p` will return the usual status fields. In particular, we can use the function `MPI_Get_count` to determine how big the message is, and we used this to be sure sufficient storage was allocated for the receive buffer for the message. If we're unable to allocate sufficient storage for the receive buffer, we can call `MPI_Abort`:

```

int MPI_Abort(
    MPI_Comm    comm         /* in */,
    int        errorcode     /* in */);

```

This is supposed to terminate all the processes in `comm` and return `errorcode` to the invoking environment, although most implementations terminate all the processes in `MPI_COMM_WORLD`.